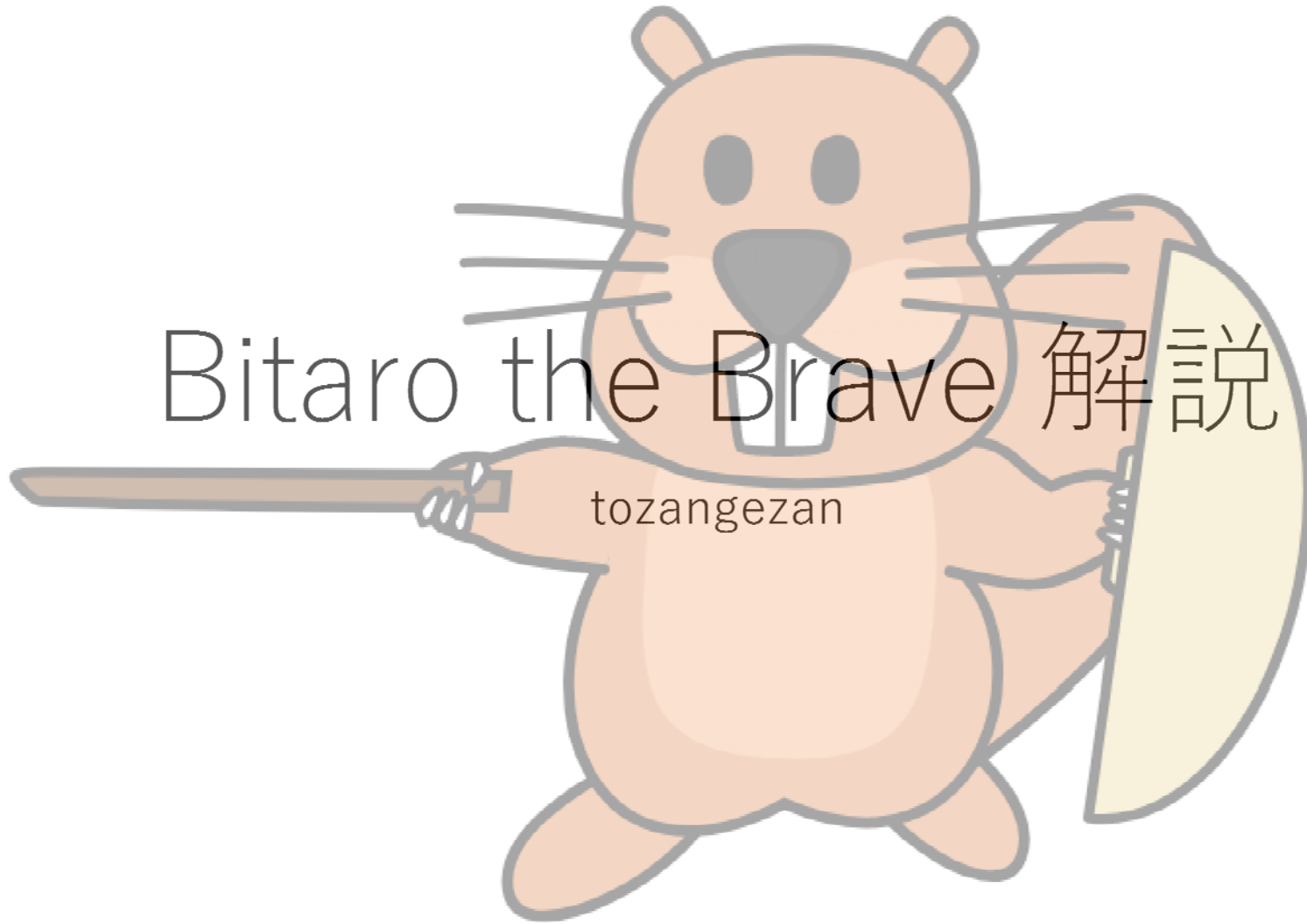


Bitaro the Brave 解説

tozangezan



問題概要

- J, O, I かなる $H \times W$ のグリッドがあたえられる。
- 左上にJ, 右上にO, 左下にI が書かれている長方形は何個？
- $H, W \leq 3,000$
- 部分点1: $H, W \leq 100$
- 部分点2: $H, W \leq 500$

簡単な解法

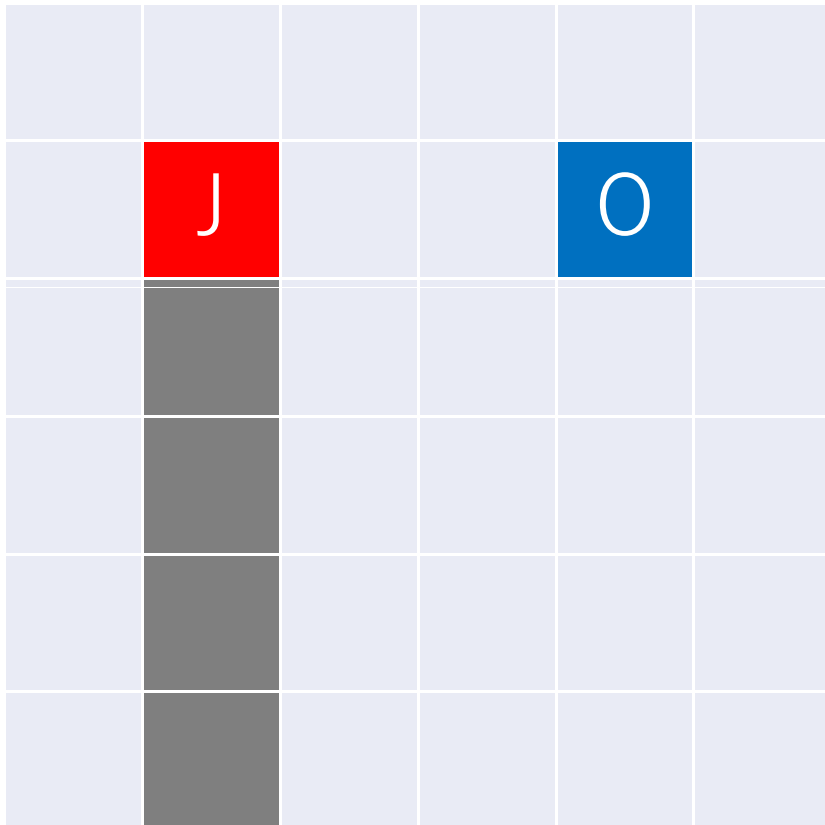
- すべての長方形領域を試してみよう！
- ```
for(int i=0;i<H;i++) // 上はどこか？
 for(int j=i+1;j<H;j++) // 下はどこか？
 for(int k=0;k<W;k++) // 左はどこか？
 for(int l=k+1;l<W;l++) // 右はどこか？
 if(s[i][k]=='J'&&s[i][l]=='O'&&s[j][k]=='I')ans++;
```

# 計算量は？

- ループが4重になっていて、それぞれ  $H$  もしくは  $W$  の定数倍回 (1だったり  $1/2$  だったり) 回っている
- 時間計算量は？  $O(H^2W^2)$
- $H, W \leq 100$  ならば大丈夫そう
- $H, W \leq 500$  ならば  $H^2W^2 = 625$  億 なので、厳しい

# うまくまとめて早くしたい

- 上の二点を決めたらどうなるかな？



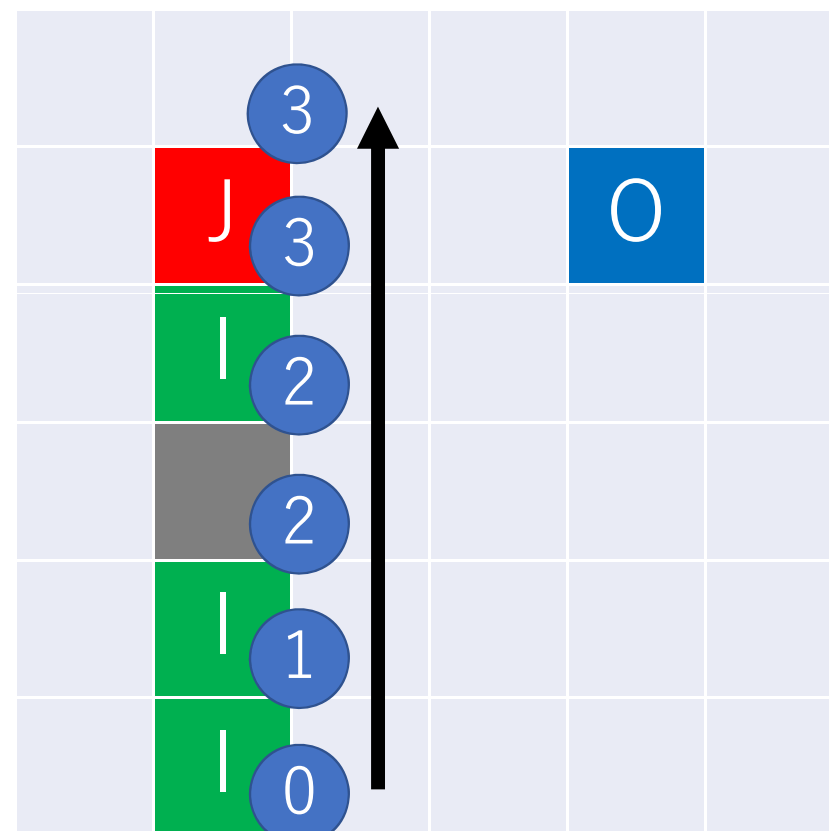
うまくまとめて早くしたい

- その列の下の方に I が何個あるかわかればよさそう

|  |   |  |  |   |  |
|--|---|--|--|---|--|
|  |   |  |  |   |  |
|  | J |  |  | O |  |
|  | I |  |  |   |  |
|  |   |  |  |   |  |
|  | I |  |  |   |  |
|  | I |  |  |   |  |

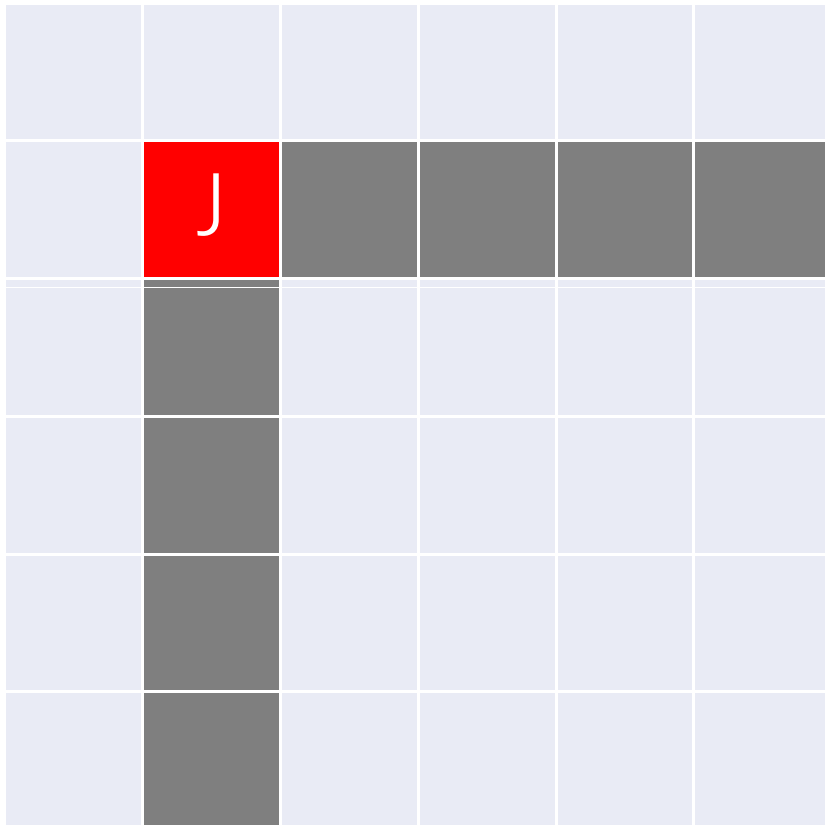
# うまくまとめて早くしたい

- 事前にそれぞれの列、行に対し、そこより下にいくつ 1 があるかを求めておこう
- すると、J と 0 を決めた瞬間条件を満たす 1 の個数が分かる
- 計算量は  $O(HW^2)$
- 小課題 2 も解けました



# もっと早くしたい

- 左上の一点を決めたらどうなるかな？





# もっと早くしたい

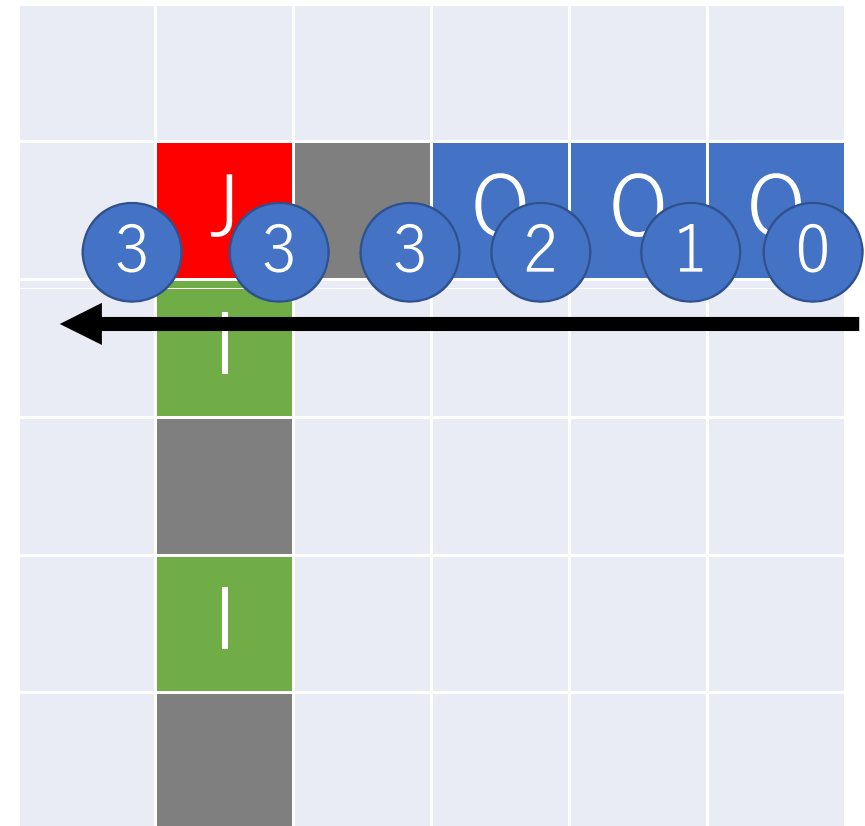
- 右に 0 が何個あるか、下に 1 に何個あるかわかれば、積を求めて合計すればよさそう

|  |   |  |   |   |   |
|--|---|--|---|---|---|
|  |   |  |   |   |   |
|  | J |  | 0 | 0 | 0 |
|  | 1 |  |   |   |   |
|  |   |  |   |   |   |
|  | 1 |  |   |   |   |
|  |   |  |   |   |   |

左の図の J を選んだ時は、6通り

# もっと早くしたい

- 小課題 2 で計算したものに、あるマスより右に 0 がいくつあるかも計算しておけばよい
- すると、J を決めた瞬間条件を満たす 0 と 1 の個数が分かる
- 掛け算すれば 0, 1 の組み合わせの個数
- もわかる
- 計算量は  $O(HW)$
- 小課題 3 も解けました

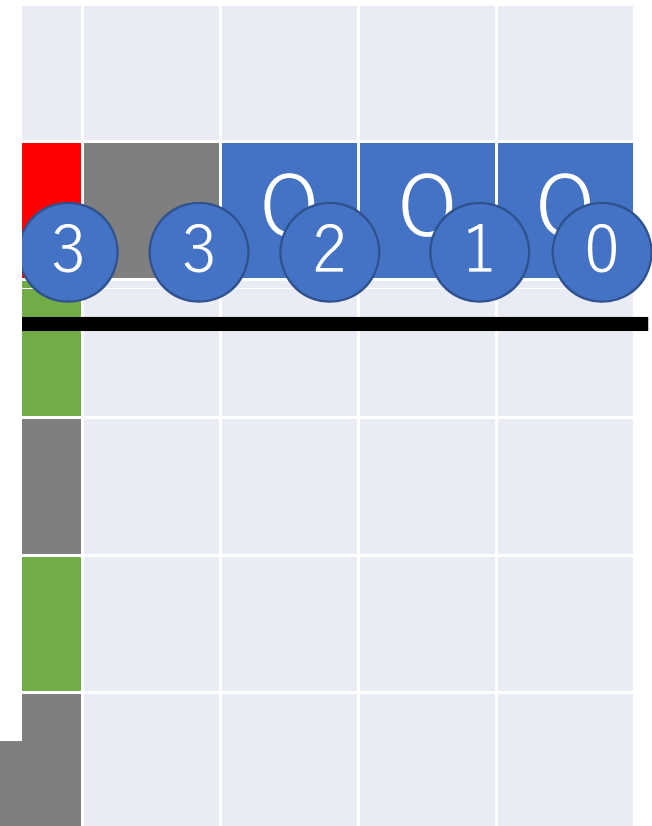


もっと早く

- 小課題 2 で計算量  
くつあるかも言
- すると、J を満  
満たす 0 と 1
- 掛け算すれば
- もわかる
- 計算量は  $O(H)$
- 小課題 3 も解



より右に 0 がい



答えは int に入るかな…？



- 左上の  $1500 \times 1500$  が全部 J, 左下が 0, 右上が 1 だったら答えは少なくとも 5兆以上
- int には入りません
- long long を使おう！

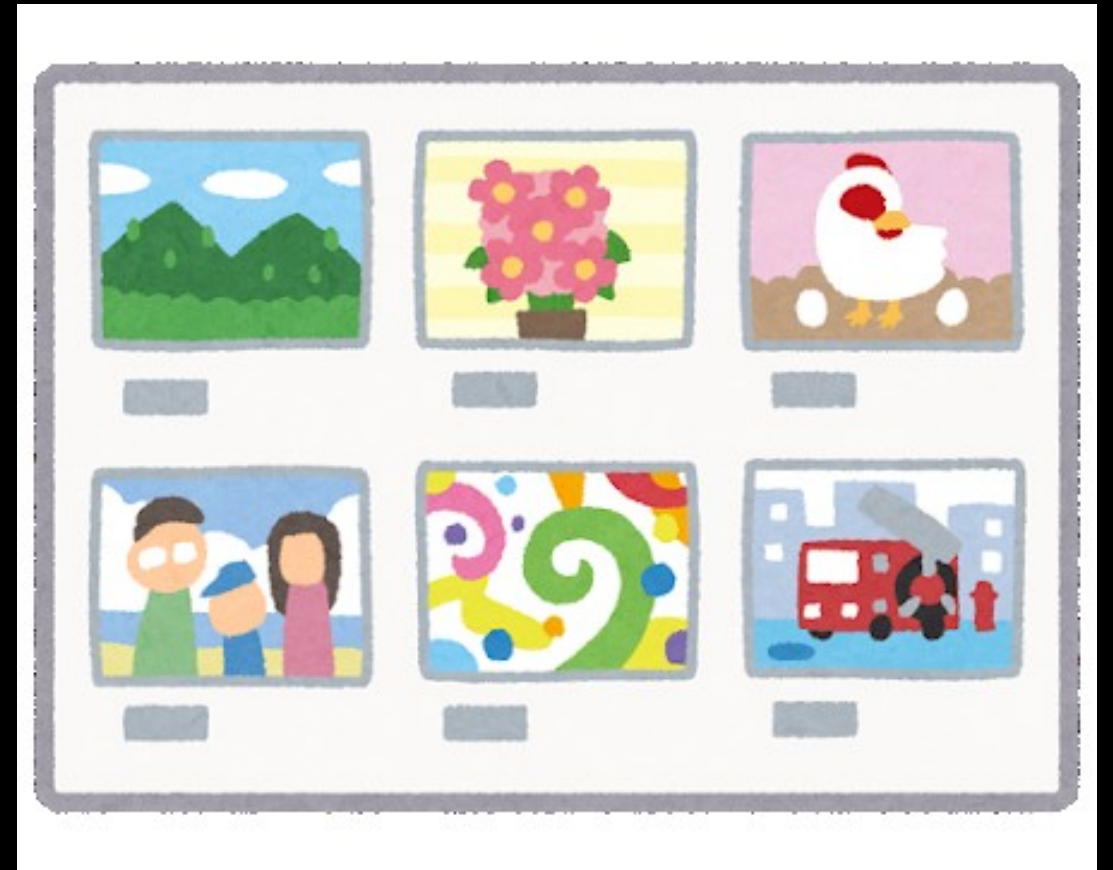
## 得点分布

| 点 | 0 | 20 | 50 | 100 |
|---|---|----|----|-----|
| 人 | 1 | 1  | 1  | 86  |



# 展覧会 (Exhibition)

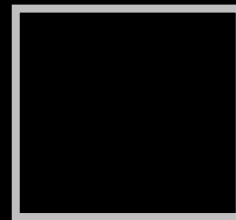
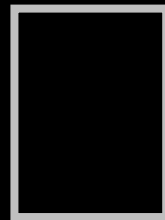
坂部 圭哉



# 問題概要

- 絵と額縁がいくつかあり、  
額縁に入れた絵を一行に並べる
- 額縁の大きさは左から広義単調増加
- 絵の価値も左から広義単調増加
- **枚数**を最大化したい

例



¥1

¥10

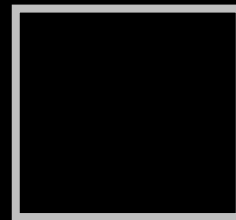
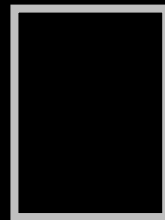
¥2

¥4

¥8



例



¥1

¥10

¥2

¥4

¥8

# 悪い例

¥4

¥8

¥10

# 悪い例

¥4

¥8

¥10

¥10

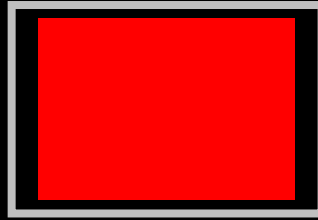
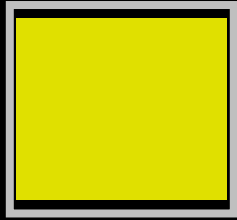
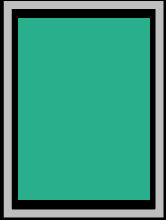
¥4

¥1

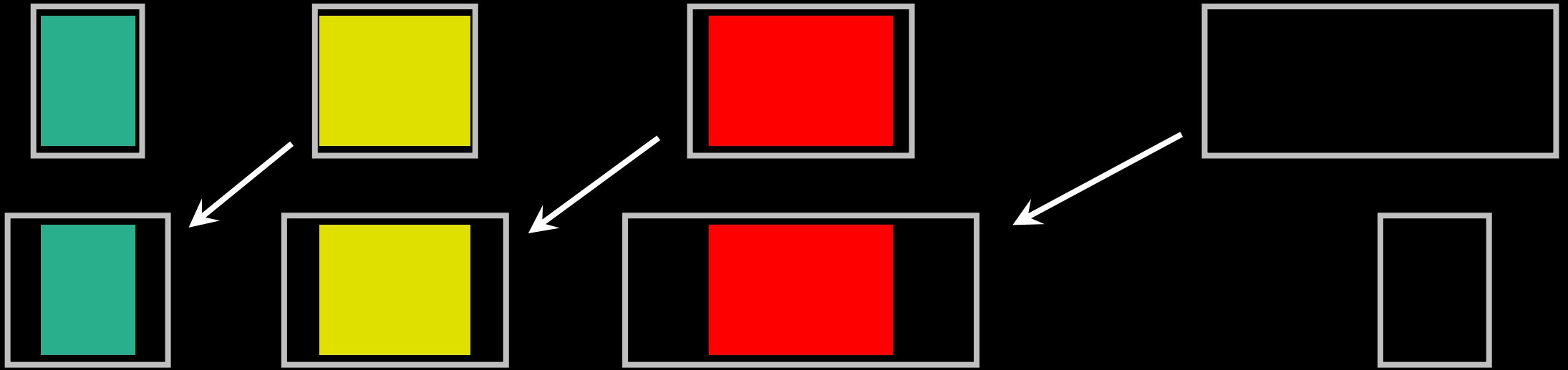
# 悪い例



# 考察(額縁)



# 考察(額縁)



額縁は、大きいほど良い。

額縁は、右から大きい順に使う。

# 考察(絵)

使う絵(の集合)を決めると  
その並べる順番は決まる

- ・ 左から、価値が小さい順
- ・ 同じ価値がある場合は、  
大きさが小さい順(逆は×)

# 解法1

- 使う絵の集合を決める( $2^N$ 通り)
- 絵・額縁を並べる順序が決まる
- 絵が額縁に入るか判断する



# 解法1

- 使う絵の集合を決める( $2^N$ 通り)
  - 絵・額縁を並べる順序が決まる
  - 絵が額縁に入るか判断する
- $O(N^2 2^N)$  くらい 小課題1(10点)

# 考察2

- すべての絵・額縁を使う順にソートしておく
- 絵  $i$  が額縁  $j$  に入って展示される場合、それより右には  $i$  番目より後の絵、 $j$  番目より後の額縁しか現れない。

# 考察2

- すべての絵・額縁を使う順にソートしておく
- 絵  $i$  が額縁  $j$  に入って展示される場合、それより右には  $i$  番目より後の絵、 $j$  番目より後の額縁しか現れない。  
→ DPができそう

# 解法2(DP)

DP[i][j]: i 番目までの絵、j 番目までの  
額縁を使うときの枚数の最大値

# 解法2(DP)

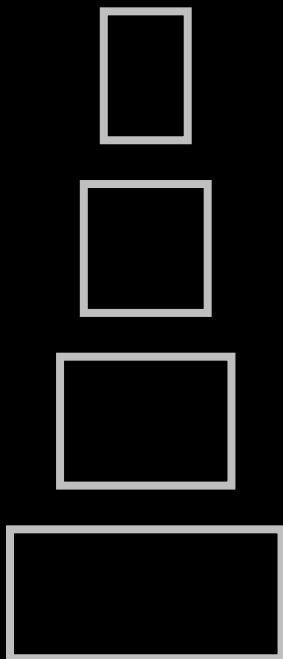
DP[i][j]: i 番目までの絵、j 番目までの額縁を使うときの枚数の最大値

$$M_{ij} = \max(DP[i-1][j], DP[i][j-1])$$

$$DP[i][j] = M_{ij} \text{ (絵 } i \text{ が額縁 } j \text{ に入らない時)}$$

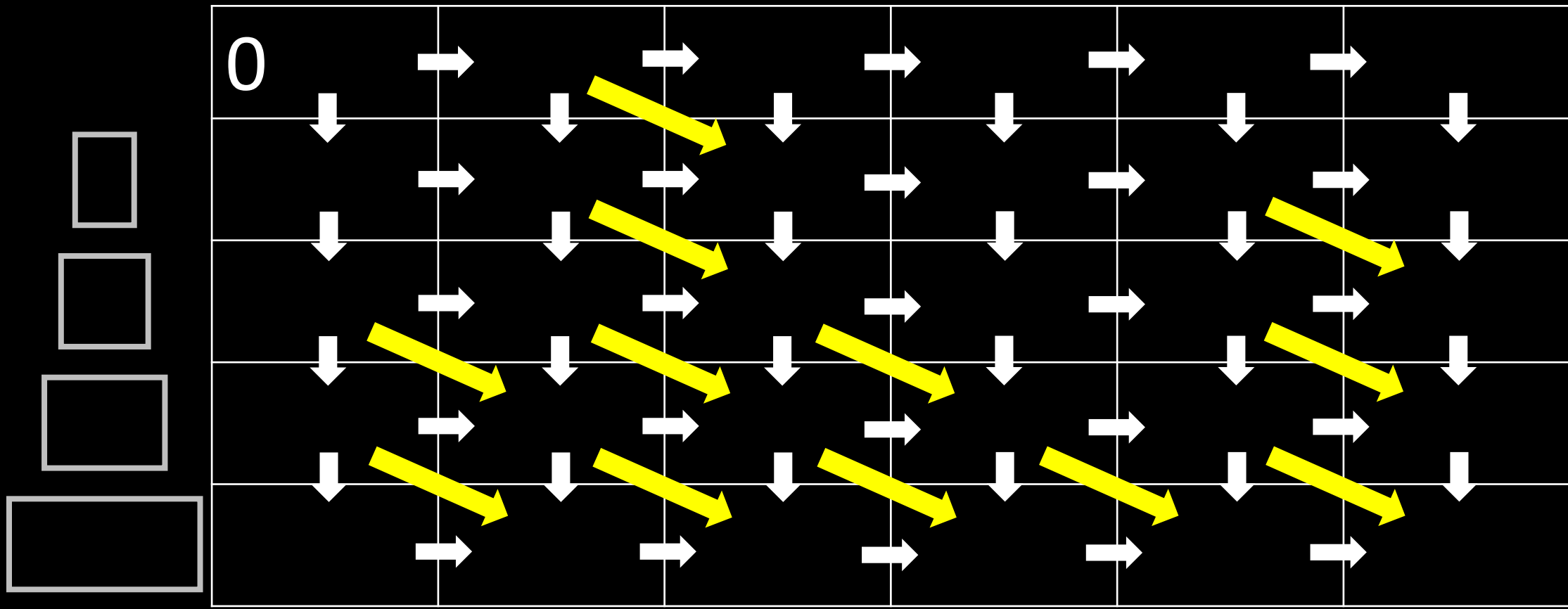
$$DP[i][j] = \max(M_{ij}, DP[i-1][j-1] + 1) \text{ (入る時)}$$

# 解法2(DP)



|   |  |  |  |  |  |
|---|--|--|--|--|--|
| 0 |  |  |  |  |  |
|   |  |  |  |  |  |
|   |  |  |  |  |  |
|   |  |  |  |  |  |
|   |  |  |  |  |  |

# 解法2(DP)



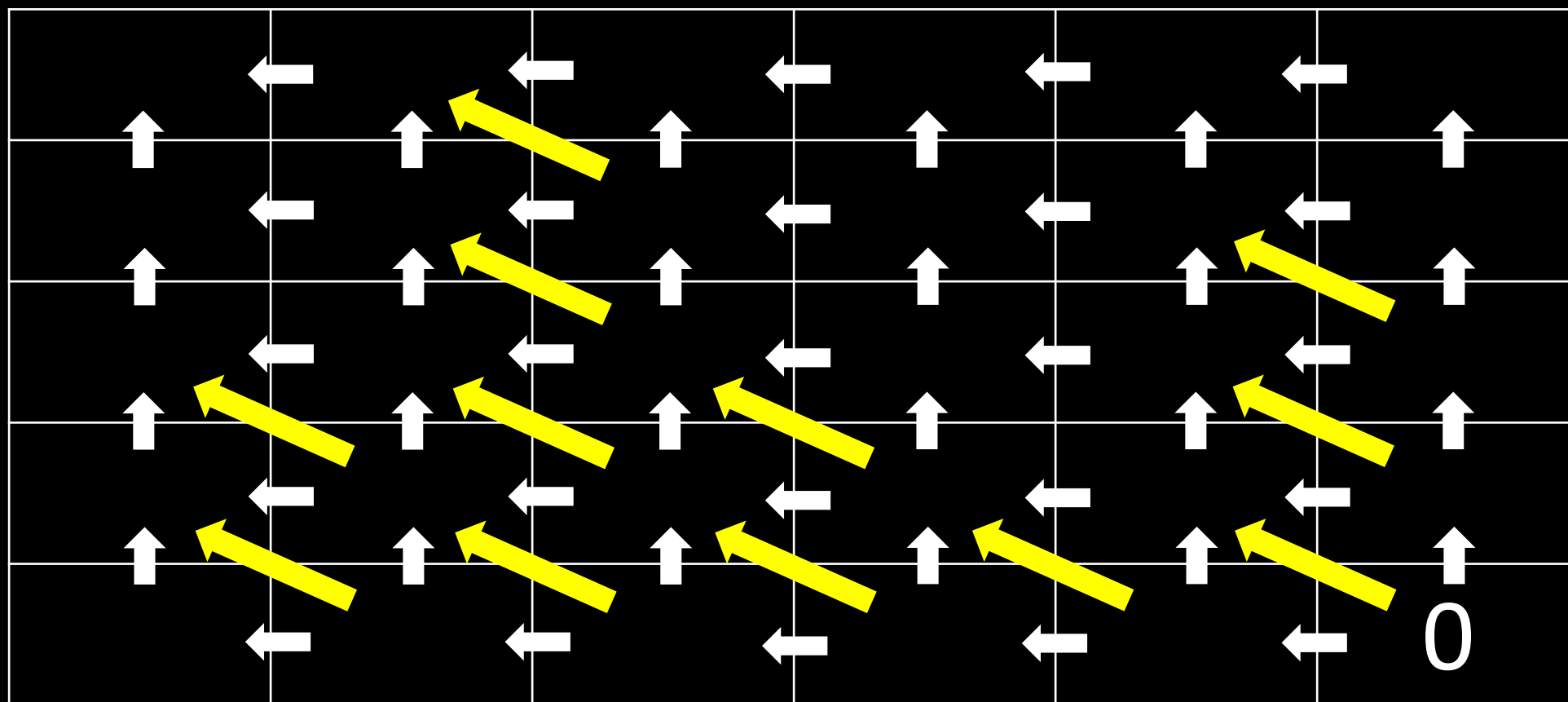
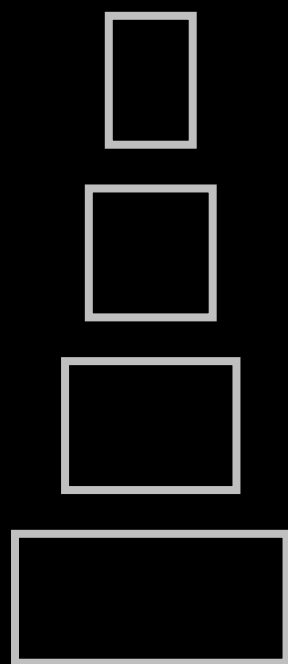
解法2(DP)

$O(NM)$

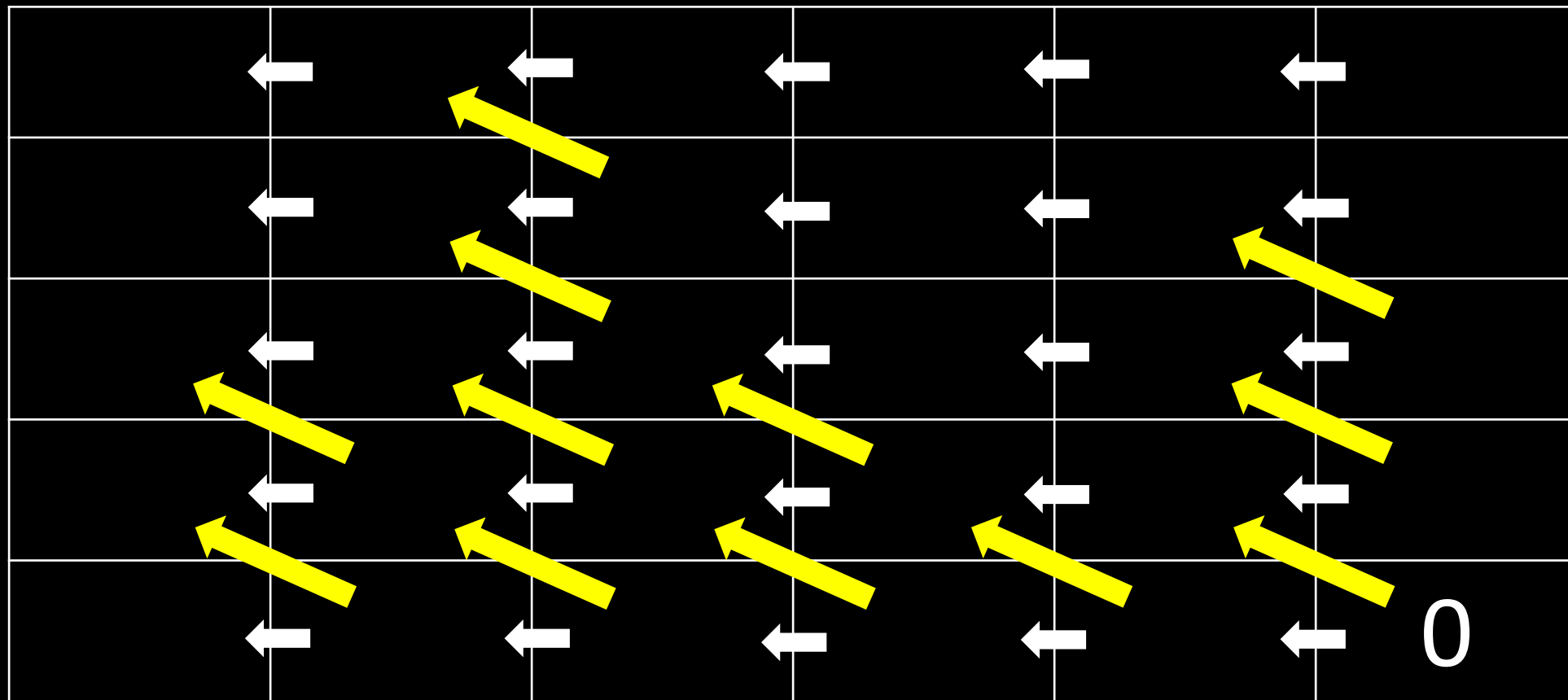
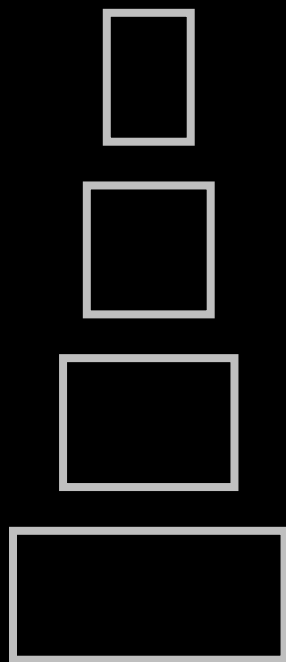
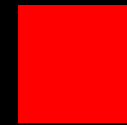
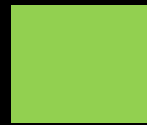
小課題2(累計50点)



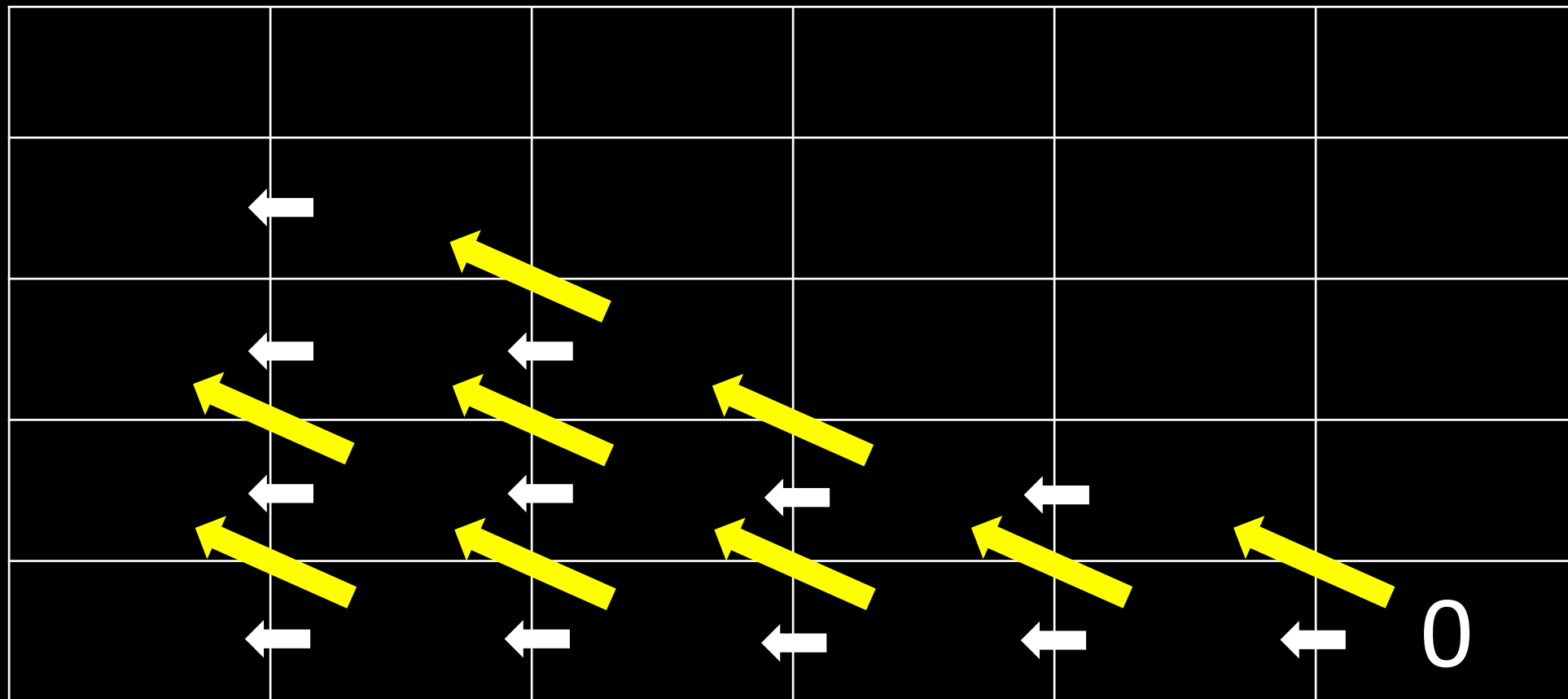
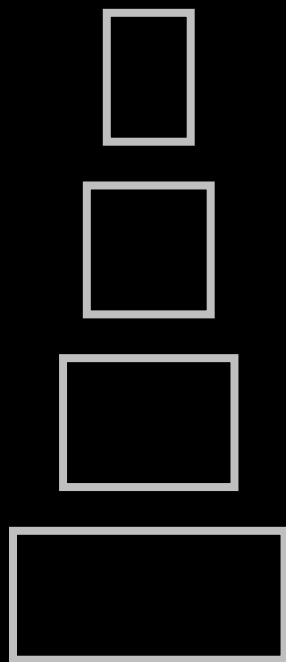
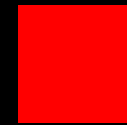
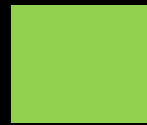
# 逆からDPしてみる...



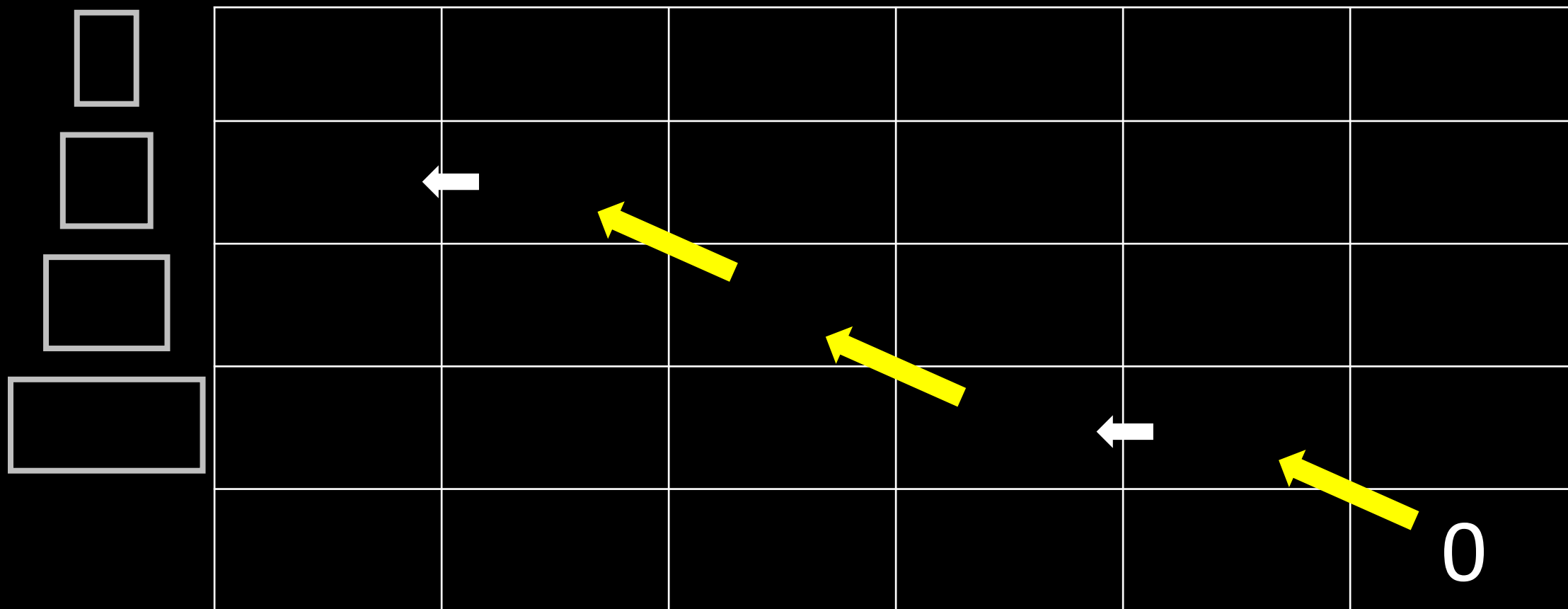
# 逆からDPしてみる...



# 逆からDPしてみる...



# この経路だけ見ればよい



# 解法3

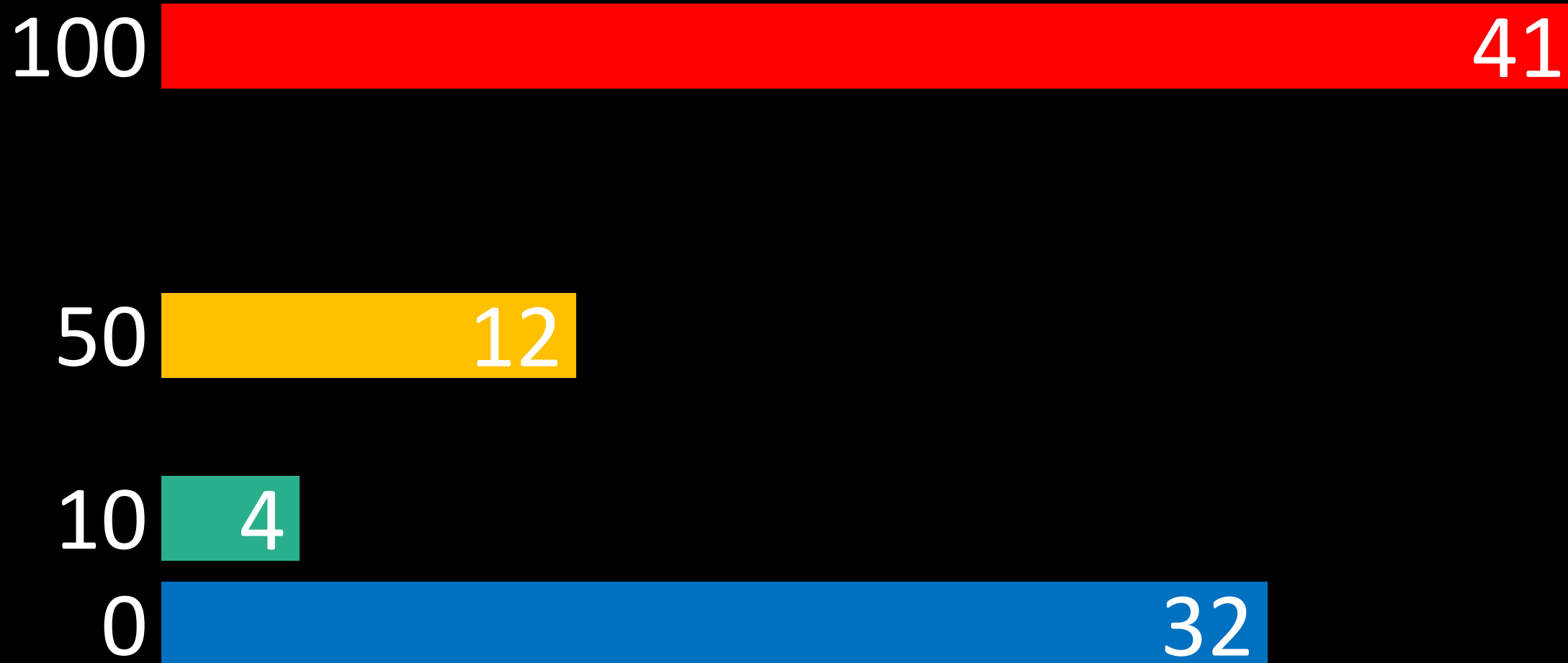
- 額縁を大きいものから、  
絵を価値の高いものから見る
- 絵が入るなら入れる

# 解法3

- 額縁を大きいものから、  
絵を価値の高いものから見る
- 絵が入るなら入れる

$O(N \log N + M \log M)$  満点

# 得点分布



# 問 3

たのしいたのしい  
たのしい家庭菜園

解説：村井



# 問題概要



(イラスト：いらすとや)

赤、緑、黄の色のいずれかの葉をつけるジョイ草がある

# 問題概要



同じ色の葉のジョイ草が隣り合うとよくないので、  
そのようなことが起きないようにしたい

# 問題概要



1 回の操作で、隣り合うジョイ草を入れ替えることができる

# 問題概要



同じ色のジョイ草が隣り合わないようにしたい

## 小課題 1 (5 点)

- $N \leq 15$ : かなり小さい
- 赤が  $r$  個、緑が  $g$  個、黄が  $y$  個の場合、並べ方は  $\frac{N!}{r!g!y!}$  通り
- $r = g = y = 5$  のときが最大で、756756 通り
- 幅優先探索 (BFS) で全探索可能

## 小課題 3 (15 点)

- $S$  の各文字は R, G のいずれかである



これを、並び替えて同じ色が隣り合わないようにしたい



並び替えた後の様子はかなり限られる：

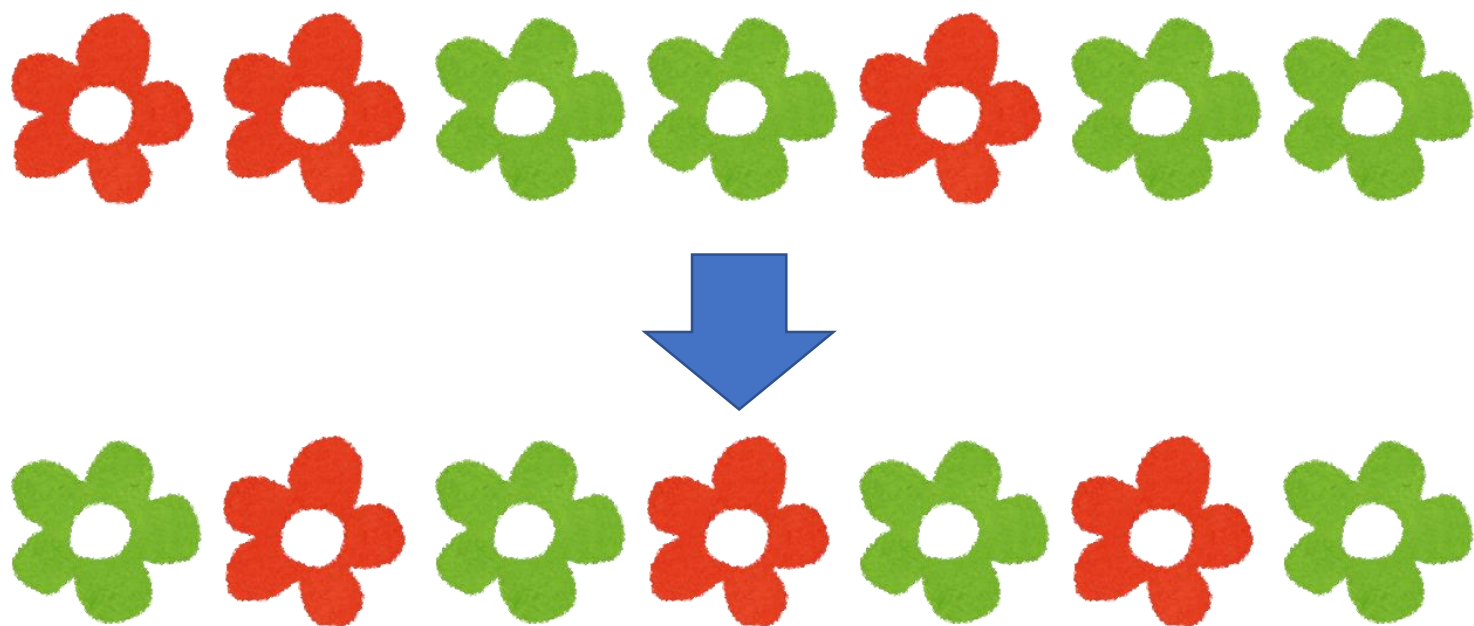
赤が最初



緑が最初

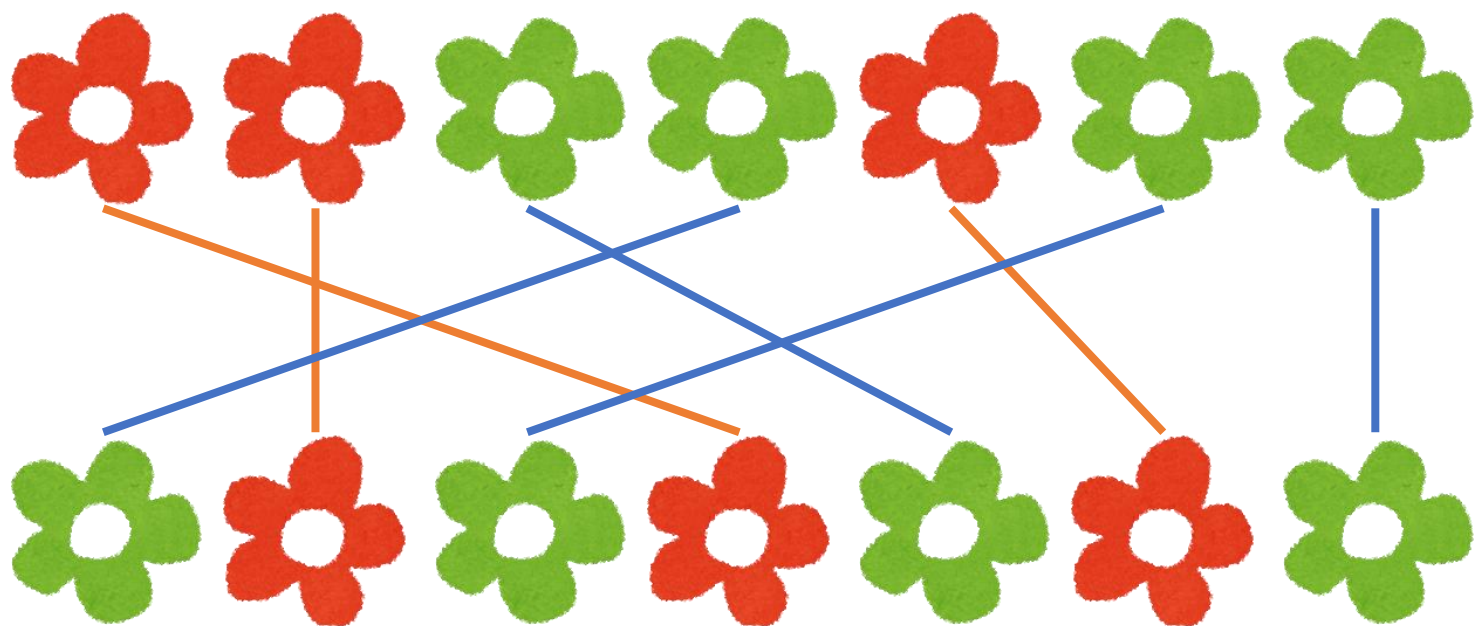


可能な目標状態が 2 通りだけなので、両方試せばよい？

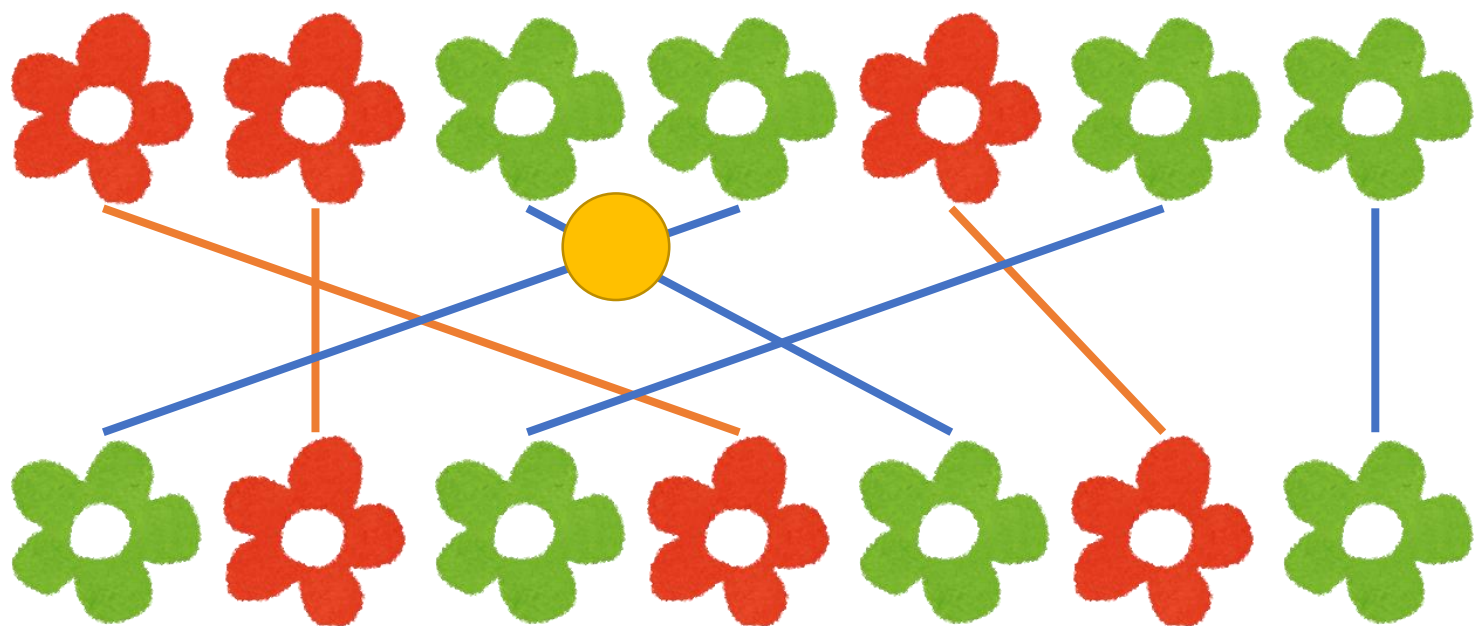


目標状態を決めたとき、  
最小の操作回数は？

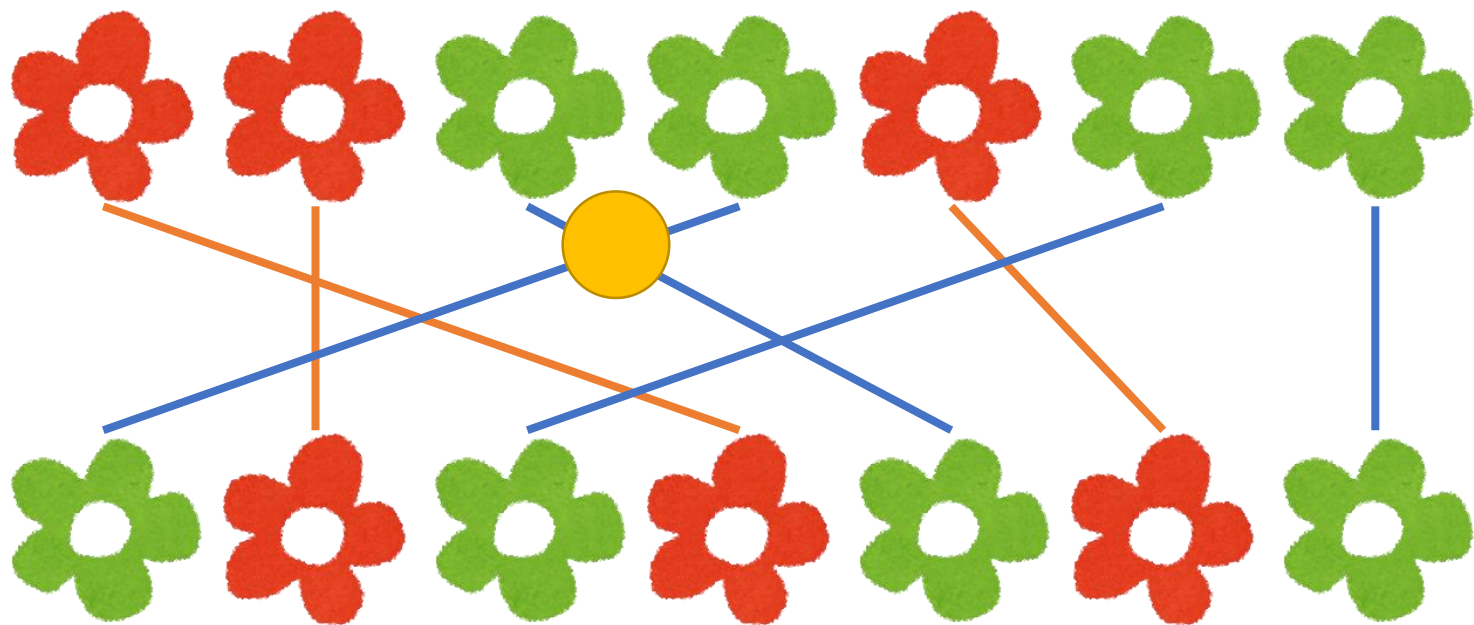




とりあえず、上と下でどのジョイ草が対応してるか考える

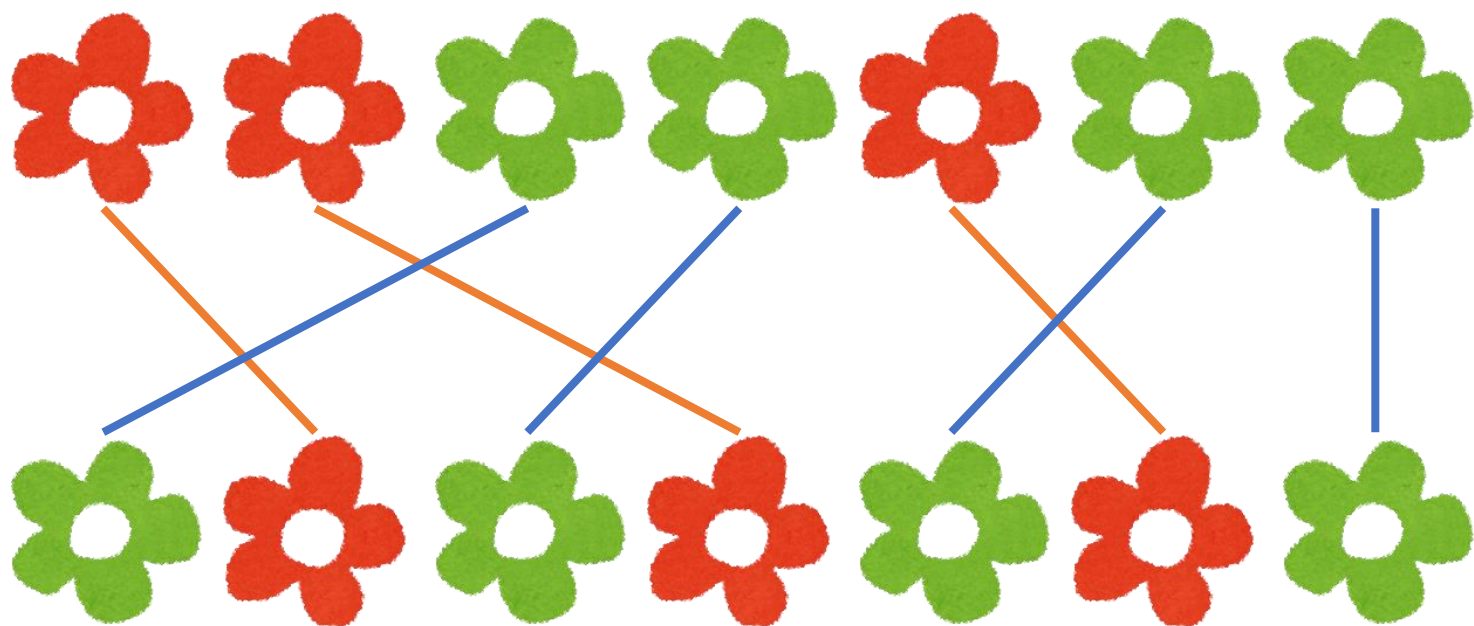


対応関係が交差している場合、その 2 つのジョ  
イ草同士を入れ替える必要がある

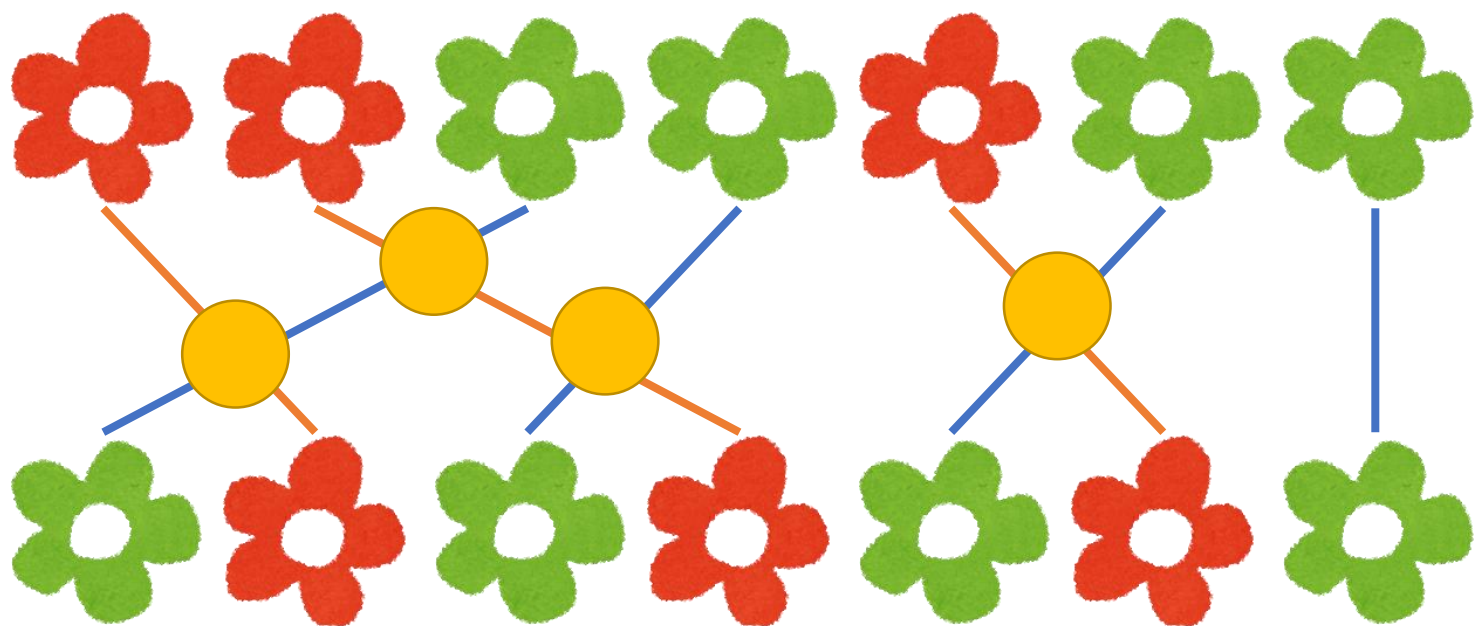


対応関係が交差している場合、その 2 つのジョイ草同士を入れ替える必要がある

でも、同じ色のジョイ草を入れ替える意味はまったくない！→最適ではない 😞

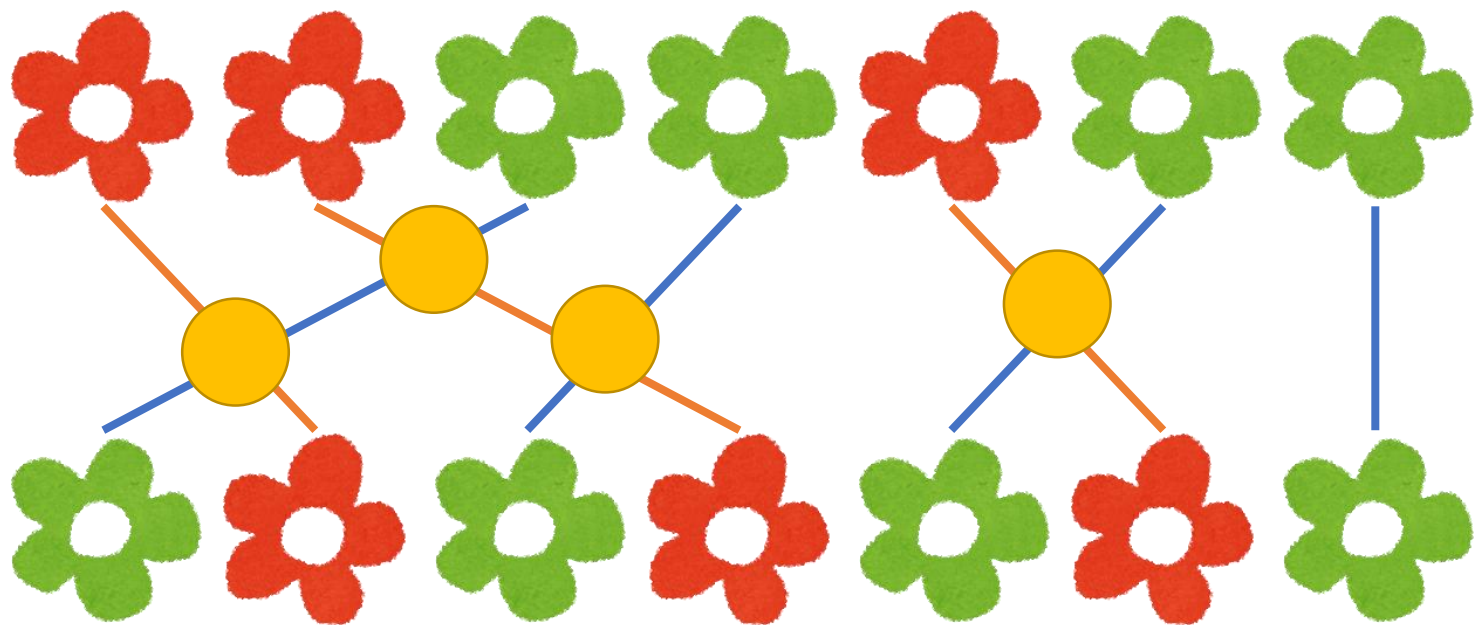


結局、同じ色の中では左から順番に対応付けるやり方だけ考えれば OK！



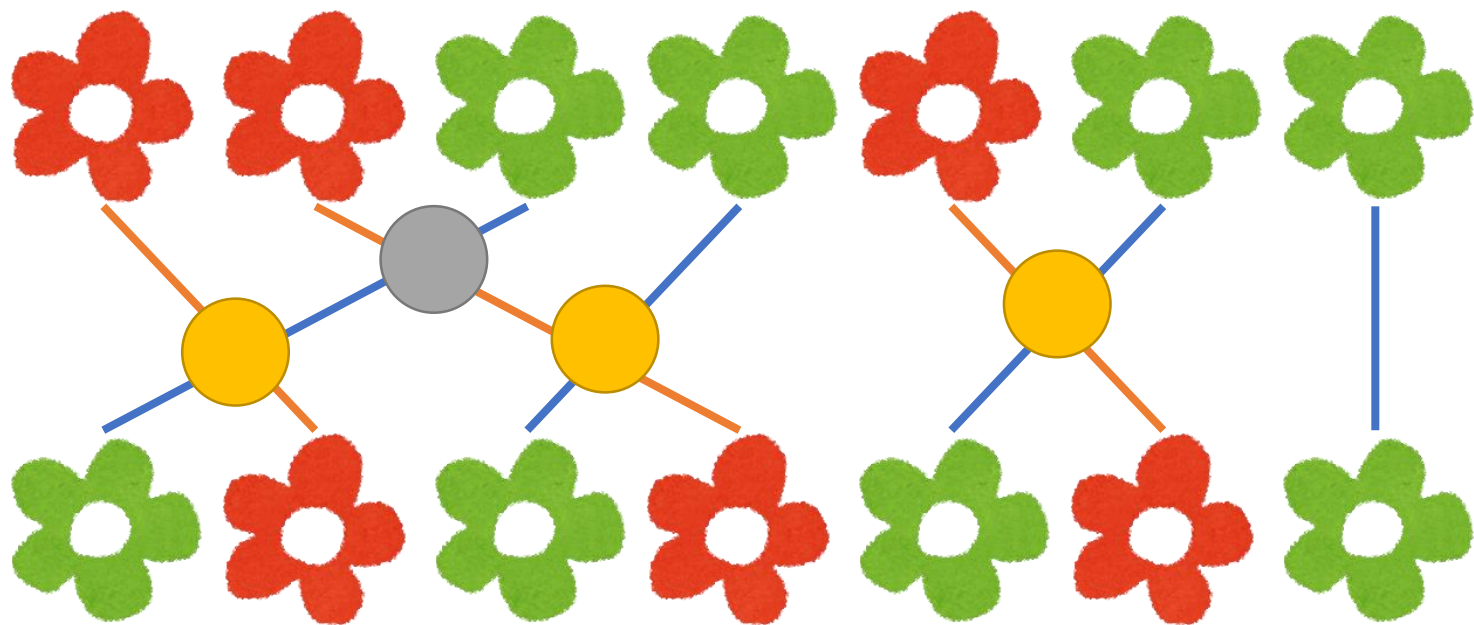
やっぱりこの交差回数くらいは最低限操作が必要  
この回数で足りる？  
→実は足りる！

なぜ？



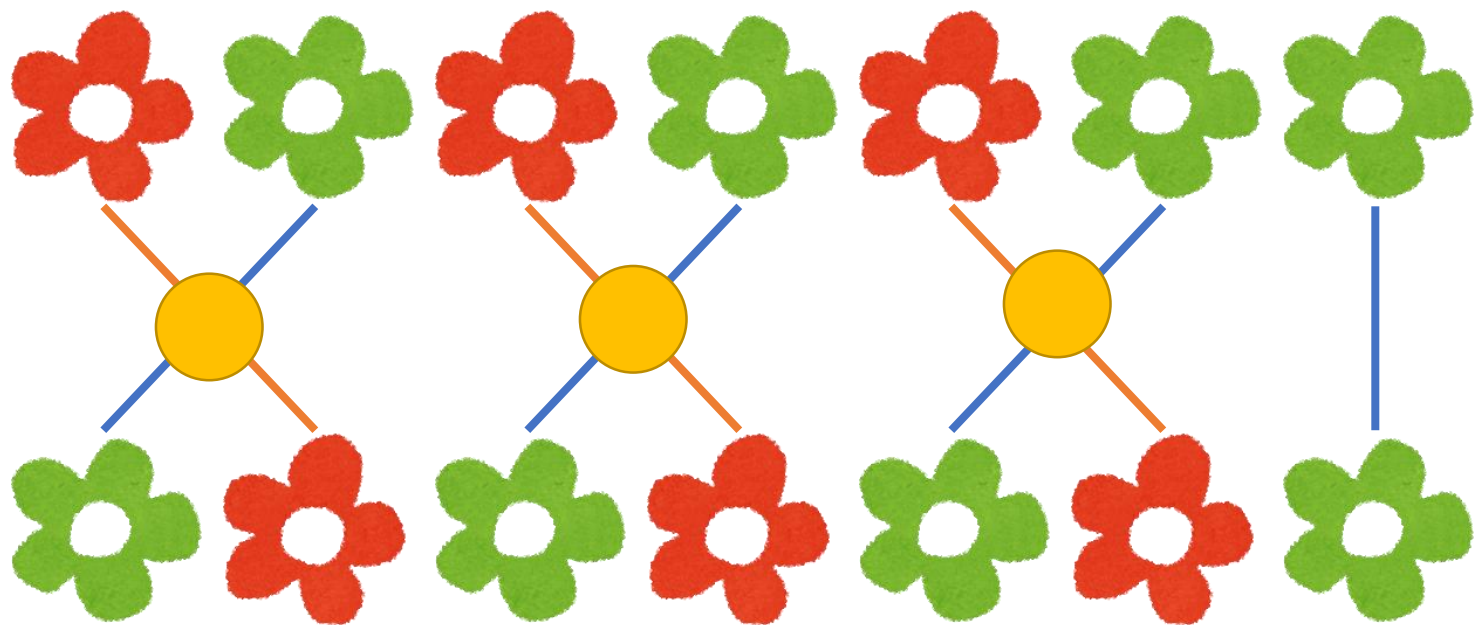
1 回の操作で、交差回数を 1 回減らすことができる

なぜ？



1 回の操作で、交差回数を 1 回減らすことができる

なぜ？



1 回の操作で、交差回数を 1 回減らすことができる

(できないとしたら、最初から正しい順に並んでいる)



## 小課題 3 の解法

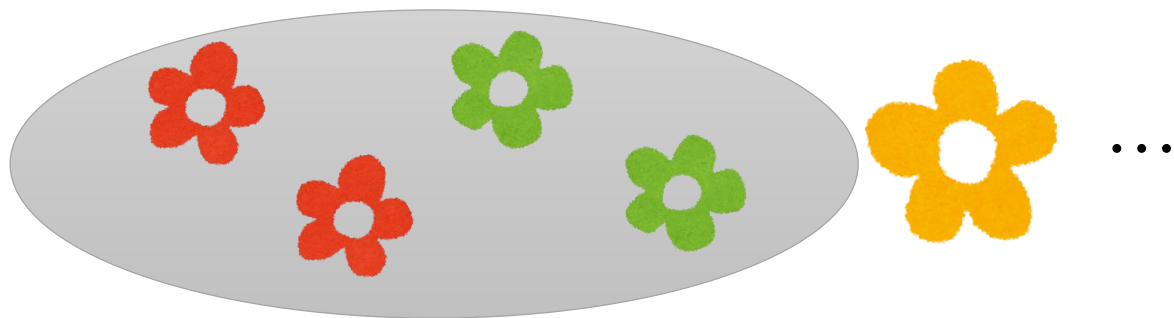
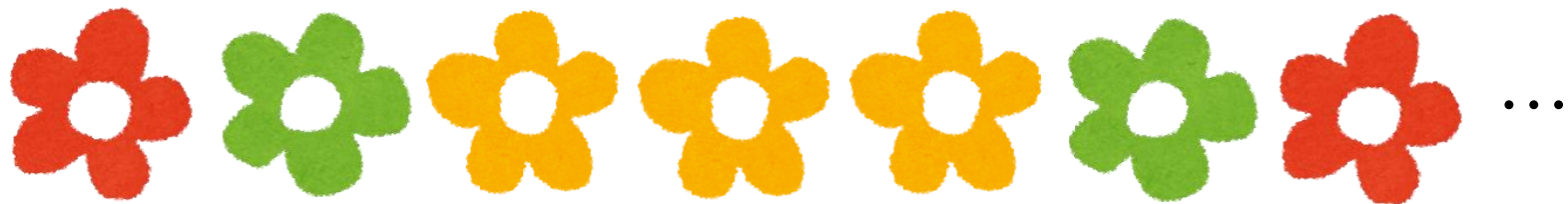
- 目標状態 2 パターンを両方試す
- そもそも赤と緑の個数が合わない場合は無理
- 個数が合っていれば、同じ色は左から順に対応させるとして、交差回数を数える

### 注意

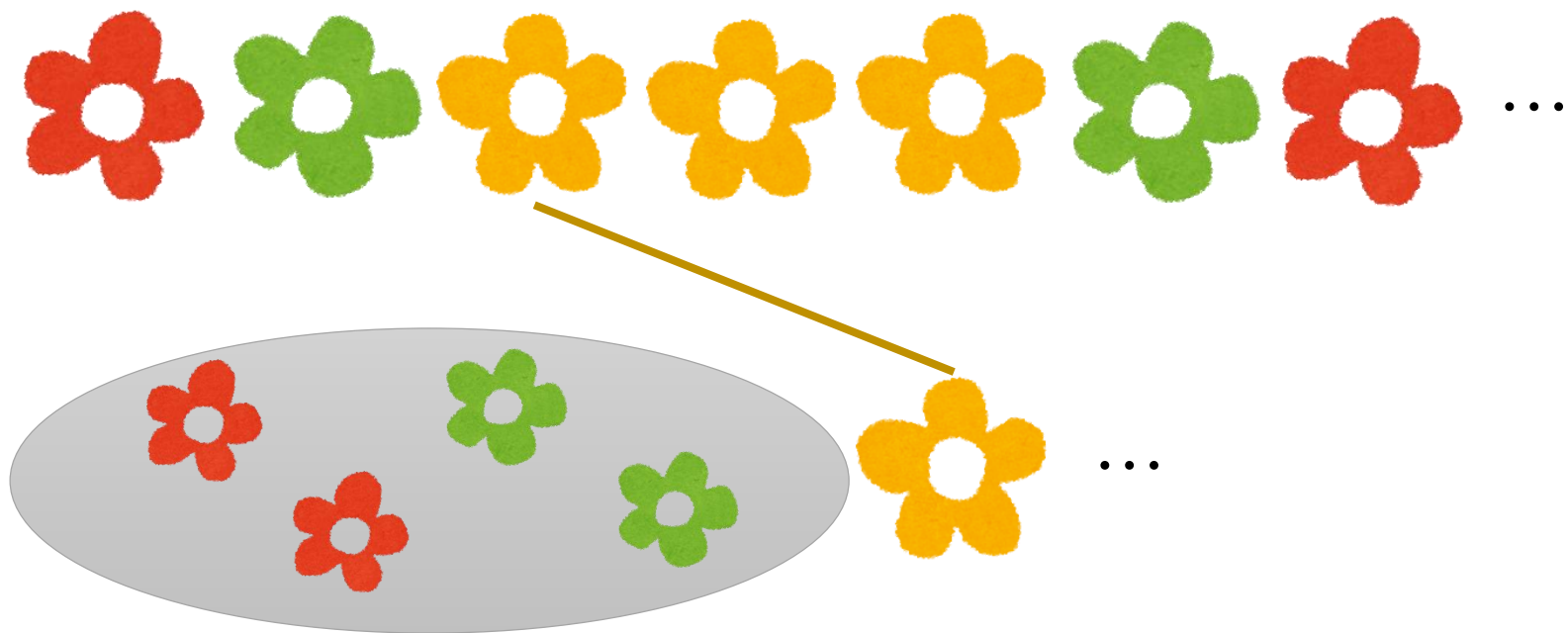
目標状態を決めてからの議論は、赤緑黄の場合にも適用可能

## 小課題 2 (55 点)

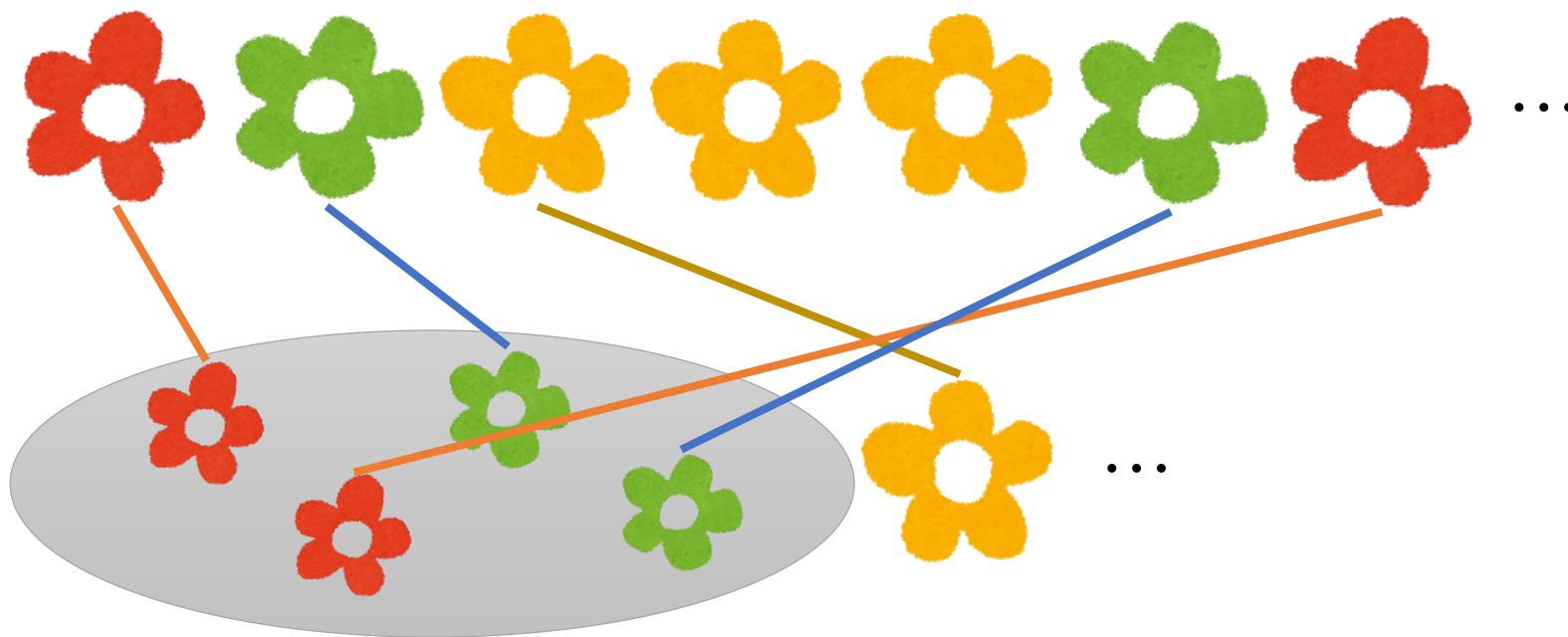
- $N \leq 60$
- 目標状態 1 個ずつ試すのはとても無理



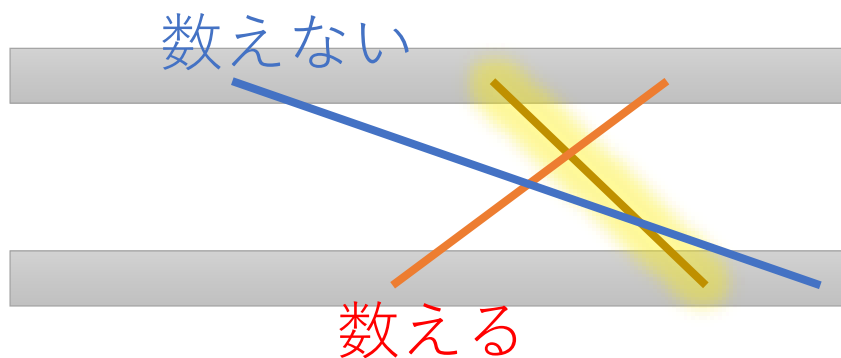
目標状態における配置を、左から順に 1 個ずつ決定していくことを考える

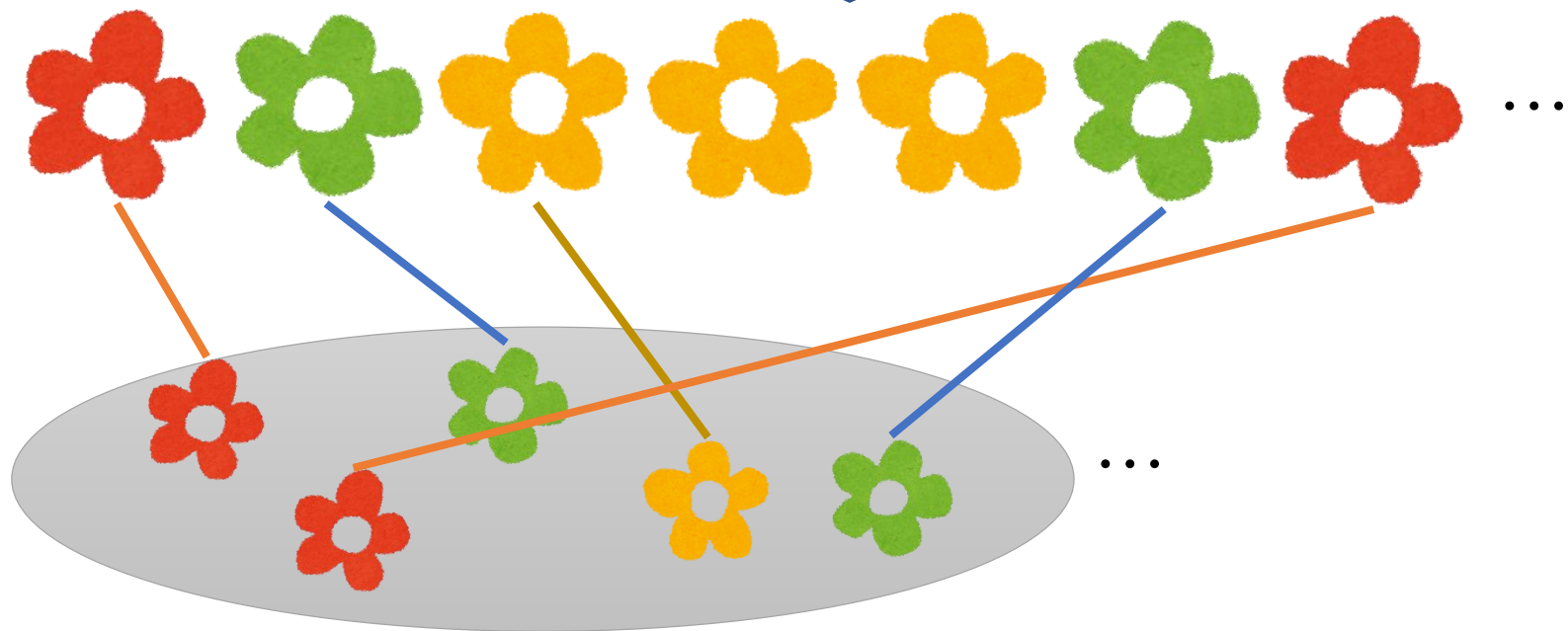
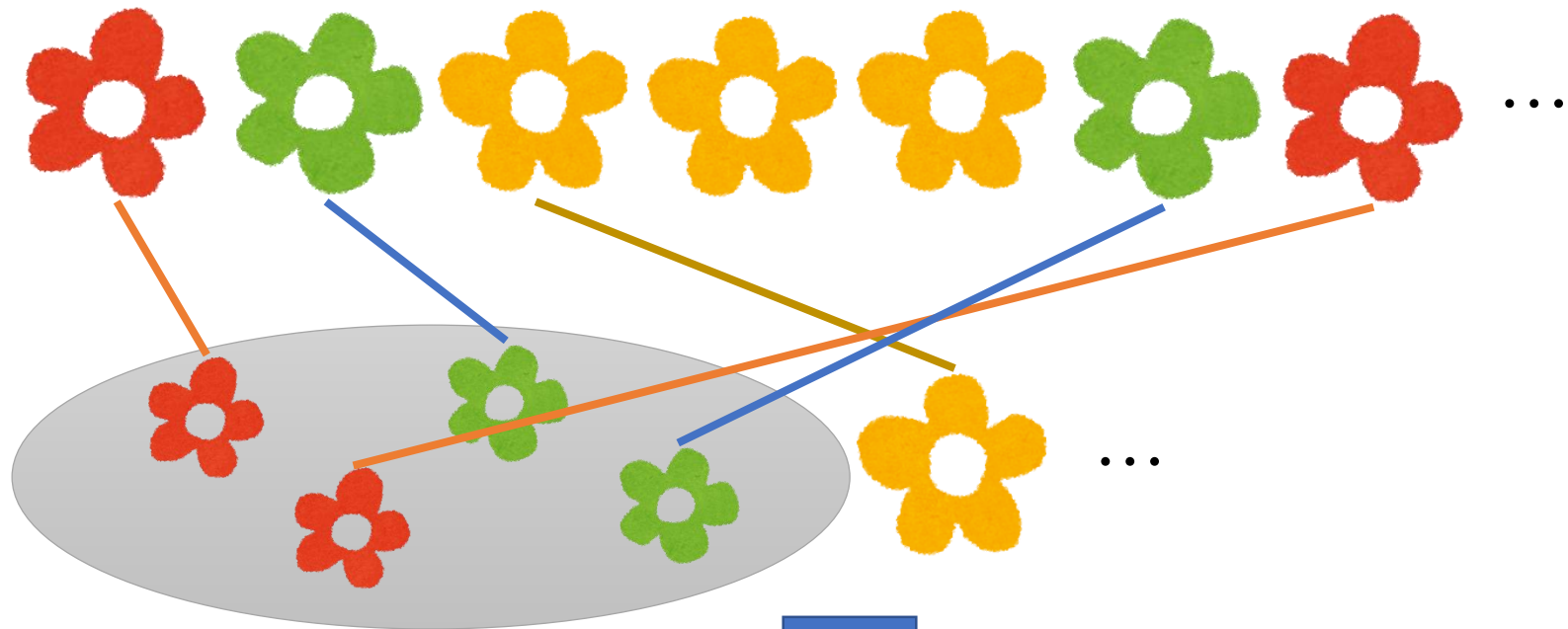


今まで使った各色の個数がわかれば、次のジョイ草が  
どれと対応するかはわかる



「使った各色個数」がわかれば、この線と交わるほかの線の個数もわかる





- よって、「赤  $r$  個、緑  $g$  個、黄  $y$  個使って、最後の色が  $c$  の場合」の、（今まで発生した）交差回数の最小値を動的計画法で計算できる
- 動的計画法の状態数は  $O(N^3)$
- 交わる線の個数を求めるのに、毎回  $O(N)$
- よって、計算量は  $O(N^4)$

## 小課題 4 (25 点)

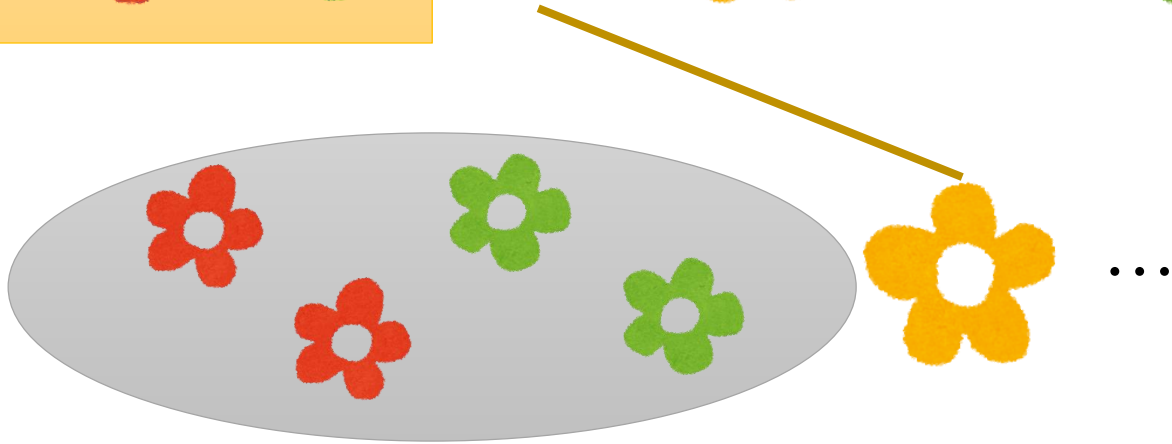
- よって、「赤  $r$  個、緑  $g$  個、黄  $y$  個使って、最後の色が  $c$  の場合」の、（今まで発生した）交差回数の最小値を動的計画法で計算できる
- 動的計画法の状態数は  $O(N^3)$
- 交わる線の個数を求めるのに、毎回  $O(N)$
- よって、計算量は  $O(N^4)$

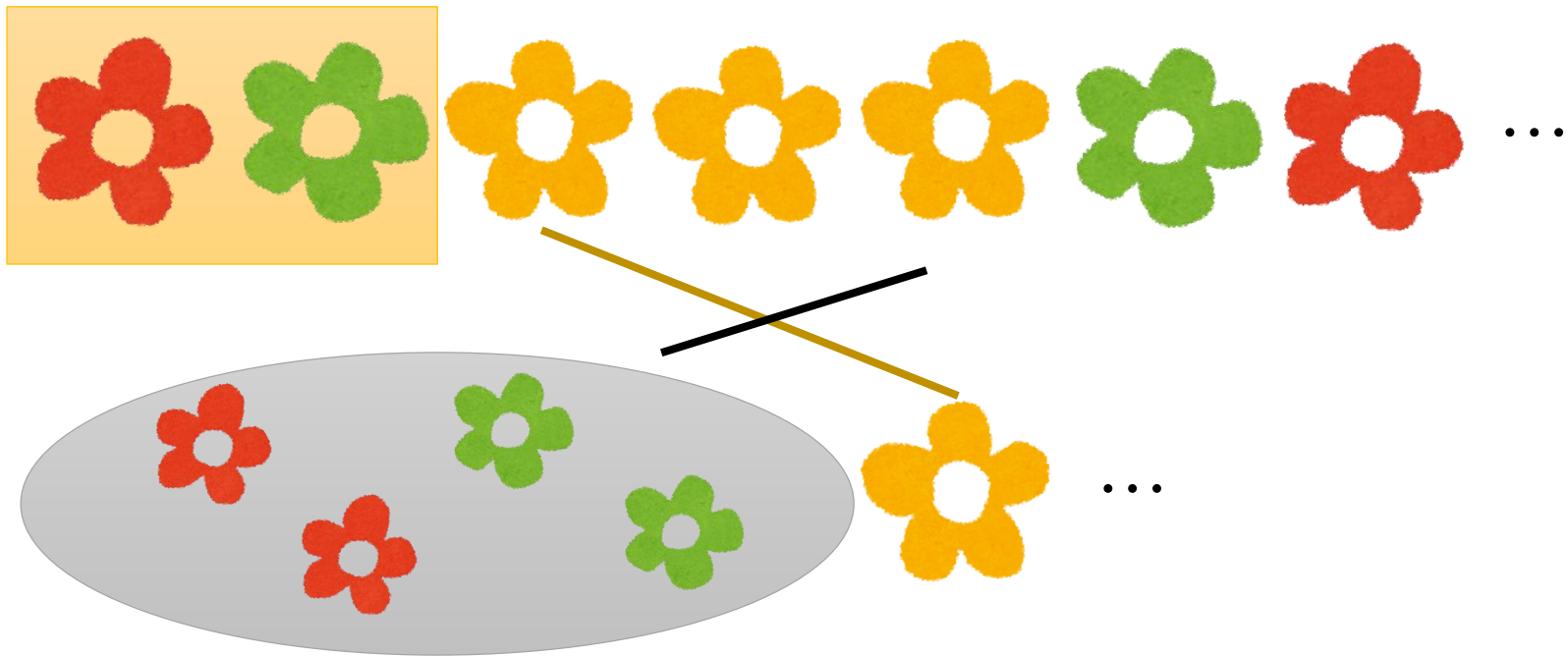


## 小課題 4 (25 点)

- よって、「赤  $r$  個、緑  $g$  個、黄  $y$  個使って、最後の色が  $c$  の場合」の、（今まで発生した）交差回数の最小値を動的計画法で計算できる
- 動的計画法の状態数は  $O(N^3)$
- 交わる線の個数を求めるのに、毎回  $O(N)$
- よって、計算量は  $O(N^4)$

これをなんとかしたい





—— と交わる線の数、

○ の中にある赤、緑の個数と、

□ の中にある赤、緑の個数からわかる

## 小課題 4 (25 点)

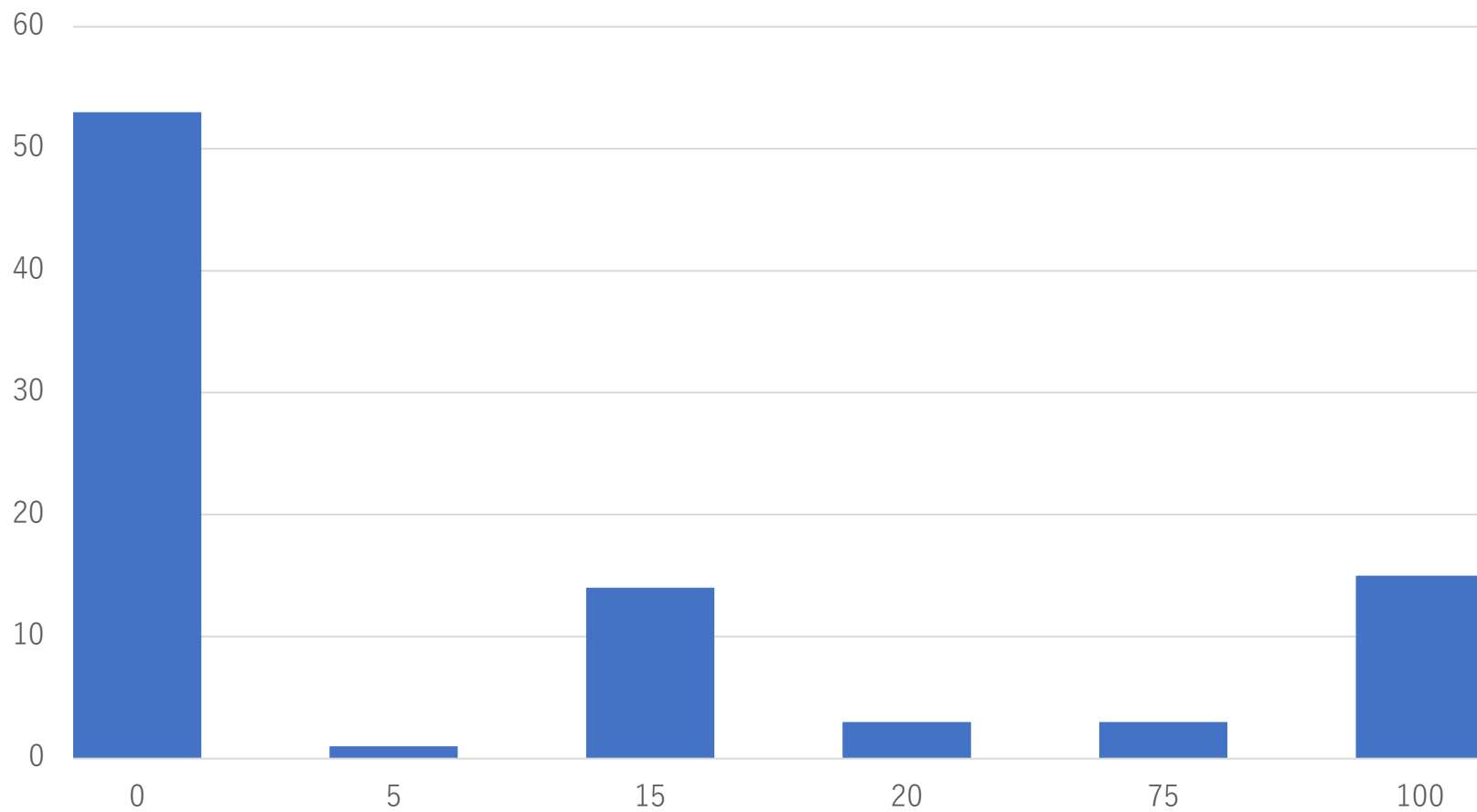
次のものを前計算しておけばよい：

- 各色、各  $i$  について、「 $i$  番目のその色のジョイ草」の位置
- 各位置について、「そこより左にある各色のジョイ草」の数

DP の遷移で、交わる線の本数を  $O(1)$  で求められる

→  $O(N^3)$  満点が得られる

# 得点分布



## 4. コイン集め (Coin Collecting)

解説：保坂

JOI 本選 2019/02/10



## 問題

マス目の  $(X_1, Y_1), (X_2, Y_2), \dots, (X_{2N}, Y_{2N})$  にあるコインを  $(1, 1), (1, 2), (2, 1), (2, 2), \dots, (N, 1), (N, 2)$  に 1 枚ずつあるようにするための最小移動回数を求めよ

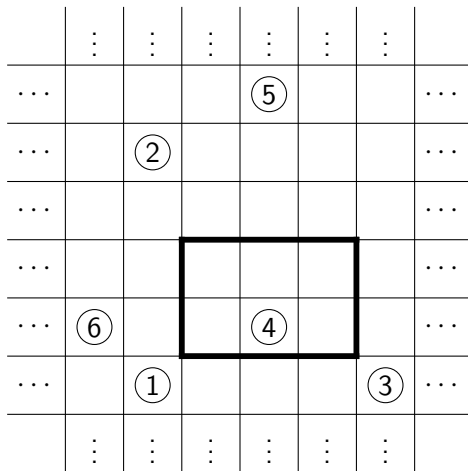
- 移動は縦横に 1 マスずつ
- 途中でコインが重なってもよい
- 座標は  $[-10^9, 10^9]$

### 小課題

1.  $N \leq 10$
2.  $N \leq 1\,000$
3.  $N \leq 100\,000$

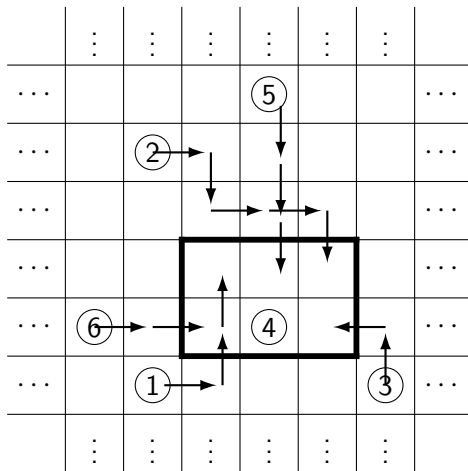


# 例





# 例



## 単純な解法

どのコインをどのマスにもっていくか決めると、あとはコインごとに独立に動かせばよい

- $(X, Y)$  から  $(x, y)$  への最小移動回数は  $|x - X| + |y - Y|$ 
  - Manhattan 距離
- 行き先の決め方は  $(2N)!$  通り
  - $20! > 2 \times 10^{18}$
  - すべて調べるのは小課題 1 でも無理



## 割り当て問題

動的計画法 (bit DP) で  $O(2^{2N}N)$  時間で解ける (小課題 1)

- 目標マスの部分集合  $S$  に対して, コイン  $1, 2, \dots, |S|$  を  $S$  のマス 1 個ずつに移動させるための最小回数を  $dp[S]$  とおく

### $O(2^{2N}N)$ 時間解法

```
foreach $S \subseteq$ (目標マスの集合): // 集合の包含の順に
 foreach $c \in$ (目標マスの集合) $\setminus S$:
 $dp[S \cup \{c\}] := \min\{dp[S \cup \{c\}], dp[S] + (\text{コイン } |S| + 1 \text{ とマス } c \text{ の距離})\}$
return $dp[\text{目標マスの集合}]$
```



## 考察

- 左右や上下にコインがすれ違うのは無駄
- 左の方のマスには左の方のコインが，右の方のマスには右の方のコインが来そう？



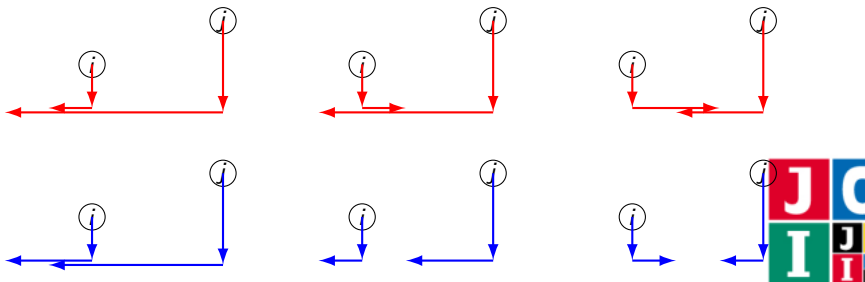
## 考察

同じ  $y$  座標に行くなら左右を保つ

以下を満たす最適解が存在する：

コイン  $i, j$  の行き先をそれぞれ  $(x_i, y_i)$ ,  $(x_j, y_j)$  とするとき、 $x_i < x_j$  かつ  $y_i = y_j$  ならば、 $x_i < x_j$

$x_i > x_j$  なら入れ替えても移動回数が増えないことを示せばよい



## 2 行に分ける

$y = 1$  にもっていくコインと  $y = 2$  にもっていくコインに分けると、あとは左右を保ったまま移動させればよい

上下の分け方をすべて試すと  $O(2^{2N}N)$  時間で解ける



## 2 行に分ける DP

動的計画法で  $O(N^2)$  時間で解ける (小課題 2)

- コインを  $X_i$  の昇順にソートしてから, コイン  $1, 2, \dots, a + b$  を  $(1, 1), (2, 1), \dots, (a, 1), (1, 2), (2, 2), \dots, (b, 2)$  に 1 個ずつ移動させるための最小回数を  $dp[a][b]$  とおく

### $O(N^2)$ 時間解法

コインを  $X_1 \leq X_2 \leq \dots \leq X_{2N}$  となるようにソート

$dp[0][0] := 0$

for  $a = 0$  to  $N$ :

for  $b = 0$  to  $N$ :

if  $a + b < 2N$ :

$dp[a + 1][b] := \min\{dp[a + 1][b],$   
 $dp[a][b] + (\text{コイン } a + b + 1 \text{ と } (a + 1, 1) \text{ の距離})\}$

$dp[a][b + 1] := \min\{dp[a][b + 1],$   
 $dp[a][b] + (\text{コイン } a + b + 1 \text{ と } (b + 1, 2) \text{ の距離})\}$

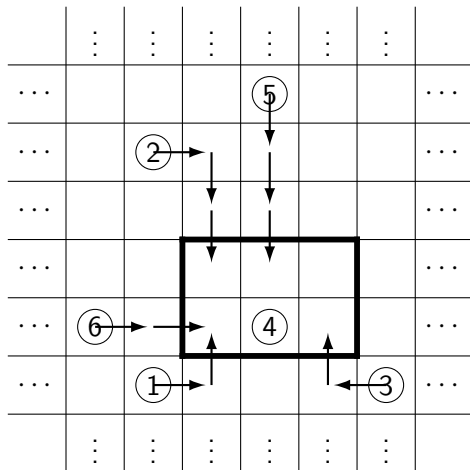
return  $dp[N][N]$



## 考察 (集める)

各コインを目標マスのうちの最寄りに移動させてしまってもよい

- $1 \leq X_i \leq N, 1 \leq Y_i \leq 2$  として考えられる



|   |   |   |
|---|---|---|
| 1 | 1 | 0 |
| 2 | 1 | 1 |

枚数





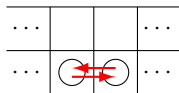
## 考察 (横移動)

### 横移動は必要最低限でよい

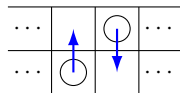
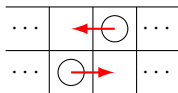
以下を満たす最適解が存在する：

各  $k = 1, 2, \dots, N-1$  について,  $1 \leq X_i \leq k$  なるコインの枚数を  $a_k$  とおくとき, コインが  $x = k$  と  $x = k+1$  の間を横に移動する回数は  $|a_k - 2k|$

- 左から右へ  $a_k - 2k$  回 or 右から左へ  $2k - a_k$  回は必要
- 左から右へも右から左へも移動しているケースは合計移動回数を増やさずになくせる



これは無駄



縦にしても同じ回数

## 考察 (横移動)

### 横移動は必要最低限でよい

以下を満たす最適解が存在する：

各  $k = 1, 2, \dots, N-1$  について,  $1 \leq x_i \leq k$  なるコインの枚数を  $a_k$  とおくとき, コインが  $x = k$  と  $x = k+1$  の間を横に移動する回数は  $|a_k - 2k|$

例 (入力例 3)

|   |   |   |   |   |
|---|---|---|---|---|
| 3 | 1 | 0 | 1 | 2 |
| 2 | 0 | 0 | 0 | 1 |

→ → ←  
3 2 1



## 考察 (縦移動)

横移動を必要最低限にしたときの縦移動の回数は？

- 縦移動はなるべく右の方で行うことにすれば，縦移動の回数が左から順番に求まる
- あるいは，左から貪欲に必要最低限の縦移動だけ行うと言ってもよい

$y = 1$  ではコインが余っていて  $y = 2$  ではコインが足りない，あるいはその逆の状況でのみ，縦移動を行えばよい



## 考察 (縦移動)

例 (入力例 3)

|   |   |   |   |   |
|---|---|---|---|---|
| 3 | 1 | 0 | 1 | 2 |
| 2 | 0 | 0 | 0 | 1 |

|   |   |   |   |   |
|---|---|---|---|---|
| 3 | 1 | 0 | 1 | 2 |
| 2 | 0 | 0 | 0 | 1 |

|   |   |   |   |   |
|---|---|---|---|---|
| 3 | 1 | 0 | 1 | 2 |
| 2 | 0 | 0 | 0 | 1 |

|   |   |   |   |   |
|---|---|---|---|---|
| 3 | 1 | 0 | 1 | 2 |
| 2 | 0 | 0 | 0 | 1 |

|   |   |   |   |   |
|---|---|---|---|---|
| 3 | 1 | 0 | 1 | 2 |
| 2 | 0 | 0 | 0 | 1 |

|   |   |   |   |   |
|---|---|---|---|---|
| 3 | 1 | 0 | 1 | 2 |
| 2 | 0 | 0 | 0 | 1 |



## 考察 (縦移動)

例

|   |   |   |   |   |
|---|---|---|---|---|
| 0 | 0 | 2 | 0 | 4 |
| 0 | 1 | 0 | 3 | 0 |

|       |   |   |   |
|-------|---|---|---|
| 0 ← 0 | 2 | 0 | 4 |
| 0 ← 1 | 0 | 3 | 0 |

|       |     |   |   |
|-------|-----|---|---|
| 0 ← 0 | ↔ 2 | 0 | 4 |
| 0 ← 1 | ← 0 | 3 | 0 |

|       |     |     |   |
|-------|-----|-----|---|
| 0 ← 0 | ↔ 2 | ← 0 | 4 |
| 0 ← 1 | ← 0 | ↔ 3 | 0 |

|       |     |     |     |
|-------|-----|-----|-----|
| 0 ← 0 | ↔ 2 | ← 0 | ↔ 4 |
| 0 ← 1 | ← 0 | ↔ 3 | 0   |

|       |     |     |     |
|-------|-----|-----|-----|
| 0 ← 0 | ↔ 2 | ← 0 | ↔ 4 |
| 0 ← 1 | ← 0 | ↔ 3 | ↓ 0 |



## まとめ

各箇所の変動回数を順に計算して  $O(N)$  時間で解ける (小課題 3)

- $y = 1, 2$  に対し,  $(x, y)$  から  $(x + 1, y)$  へ横移動するコインの枚数  $d_y$  (負の場合は逆向き) を管理する

### $O(N)$ 時間解法

$answer :=$  各コインを最寄りの目標マスに移動させ, そのときの移動回数

$d_1 := 0, d_2 := 0$

for  $x = 1$  to  $N$ :

$d_1 := d_1 + ((x, 1)$  にあるコインの枚数)  $- 1$

$d_2 := d_2 + ((x, 2)$  にあるコインの枚数)  $- 1$

if  $d_1 > 0$  and  $0 > d_2$ :

$t := \min\{d_1, -d_2\}$  //  $(x, 1)$  から  $(x, 2)$  へ  $t$  回縦移動

$answer := answer + t, d_1 := d_1 - t, d_2 := d_2 + t$

if  $d_1 < 0$  and  $0 < d_2$ :

$t := \min\{-d_1, d_2\}$  //  $(x, 2)$  から  $(x, 1)$  へ  $t$  回縦移動

$answer := answer + t, d_1 := d_1 + t, d_2 := d_2 - t$

$answer := answer + |d_1|$  //  $(x, 1)$  と  $(x + 1, 1)$  の間を  $|d_1|$  回横移動

$answer := answer + |d_2|$  //  $(x, 2)$  と  $(x + 1, 2)$  の間を  $|d_2|$  回横移動

return  $answer$

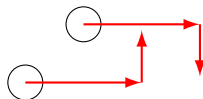


## 注意

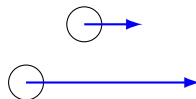
同じ  $y$  座標に行かないときは左右を保つべきとは限らない

|   |   |   |   |
|---|---|---|---|
| 1 | 2 | 0 | 1 |
| 4 | 0 | 0 | 0 |

|               |       |   |
|---------------|-------|---|
| 1             | 2 → 0 | 1 |
| 4 → 0 → 0 → 0 |       |   |



これはダメ



こっちのほうがよい

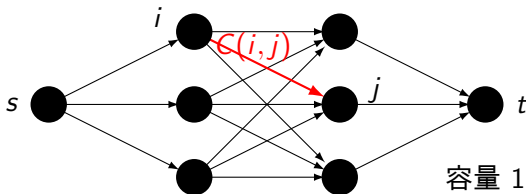
## 参考：最小費用流

### 割り当て問題

$n \times n$  のコスト行列  $C(i, j)$  が与えられたとき,  $1, 2, \dots, n$  の並べ替え  $p_1, p_2, \dots, p_n$  であって,  $C(1, p_1) + C(2, p_2) + \dots + C(n, p_n)$  が最小になるものを求めよ

小課題 1 の解説のように  $O(2^n n)$  時間で解けるが, 最小費用流あるいは Hungarian algorithm を用いると  $O(n^3)$  時間で解けることが知られている

- 小課題 2 ( $n \leq 2000$ ) は無理



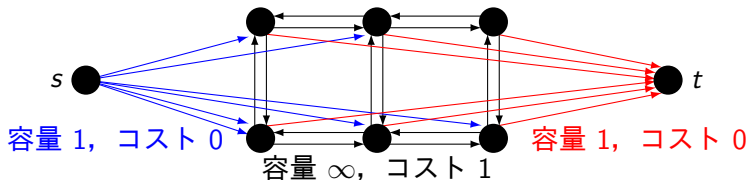


## 参考：最小費用流

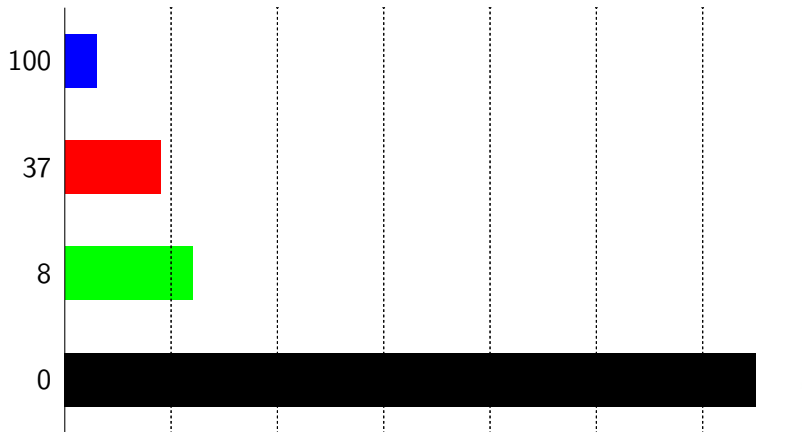
この問題ではコストが特殊なので，グラフの作り方を工夫すると  $O(N)$  頂点  $O(N)$  辺のグラフでの最小費用流に落とせる

- 最短路反復法で  $O(N^2 \log N)$  時間となり，ある程度高速な実装なら小課題 2 までは通る

|   |   |   |
|---|---|---|
| 1 | 1 | 0 |
| 2 | 1 | 1 |



# 得点分布



# 珍しい都市 解説

maroon

# 得点分布

- 小課題①の正答者      ??人
- 小課題②の正答者      ??人
- 小課題③の正答者      ??人
- 小課題④の正答者      ??人

# 問題概要

- $N$  頂点の木がある
- 頂点  $x$  から見て、距離が unique である頂点を珍しい頂点とする
- 各頂点から見て、珍しい都市についてのラベルの種類数を答える

## 小課題①

- $N \leq 2000$
- 各頂点からDFSとかBFSをしてください
- $O(N^2)$
- 4 点

# 得点分布

- 小課題①の正答者      ?1人
- 小課題②の正答者      ??人
- 小課題③の正答者      ??人
- 小課題④の正答者      ??人

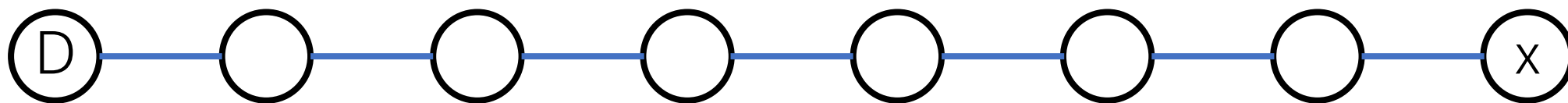
# 考察

- 頂点  $x$  から見て距離最大の頂点を一つとって  $D$  とおく
- 珍しい頂点は、存在するなら  $D-x$  パス上にある



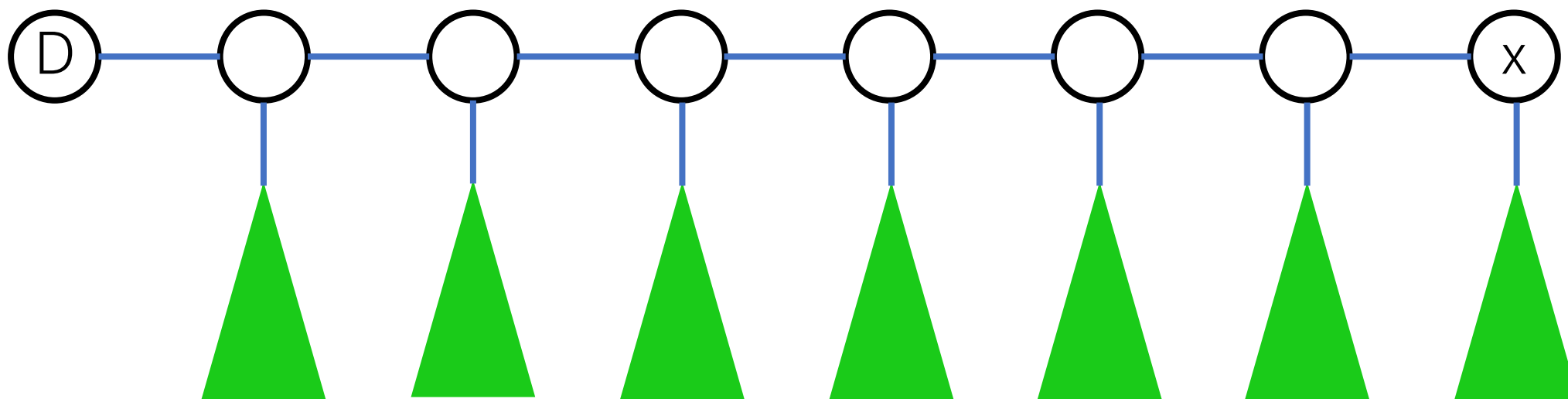
# 考察

- D-x パスに注目する



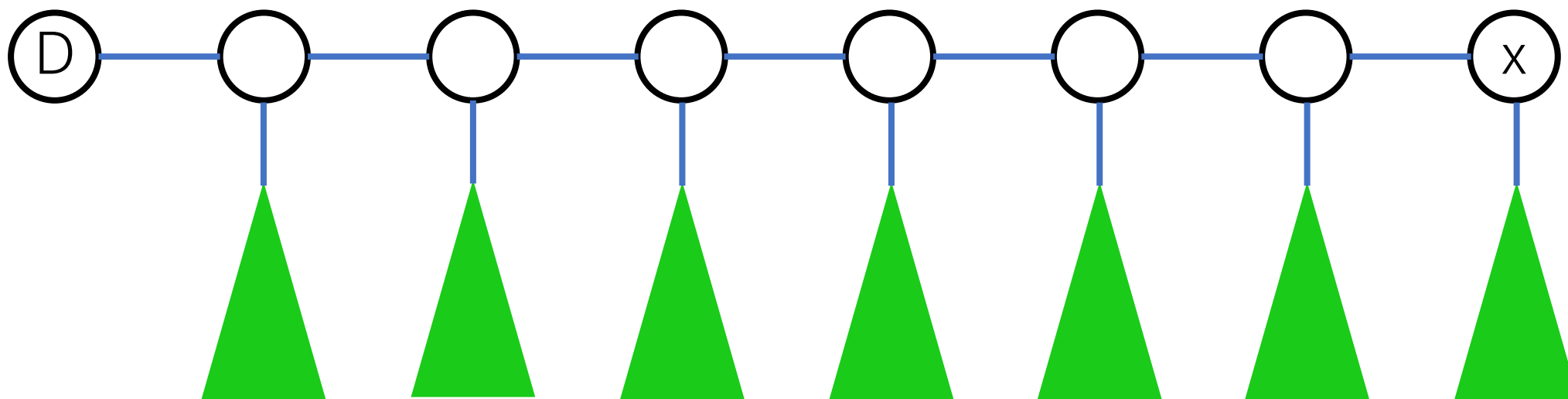
# 考察

- 途中で枝分かれしてた部分木を考える



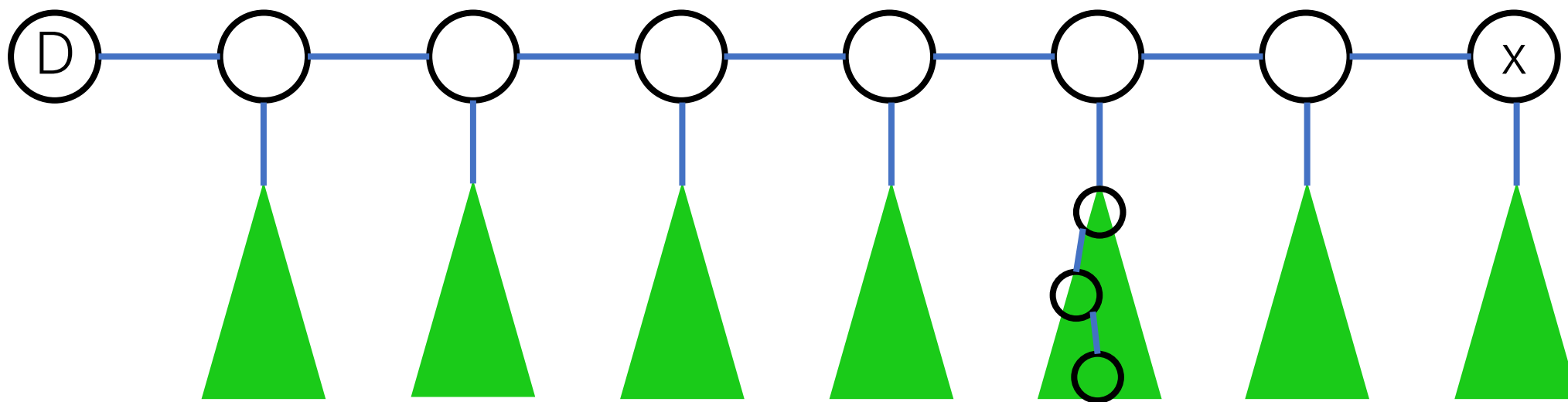
# 考察

- 枝分かれた部分木の最大深さの分だけ、珍しい都市の候補が消える



# 考察

- 枝分かれた部分木の最大深さの分だけ、珍しい都市の候補が消える

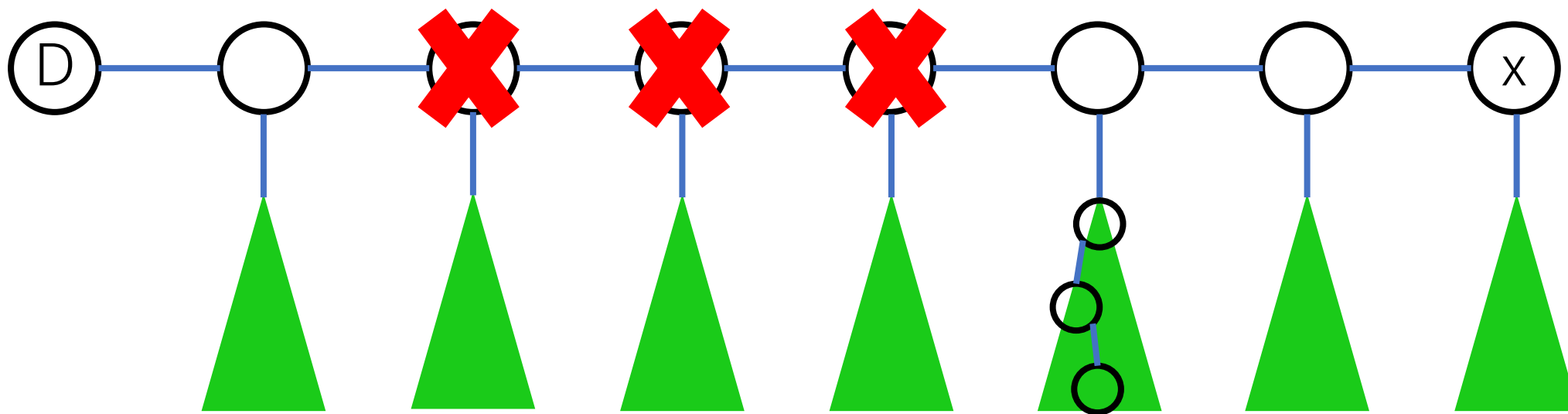


例えばここに長さ3のパスがあれば

# 考察

- 枝分かれた部分木の最大深さの分だけ、珍しい都市の候補が消える

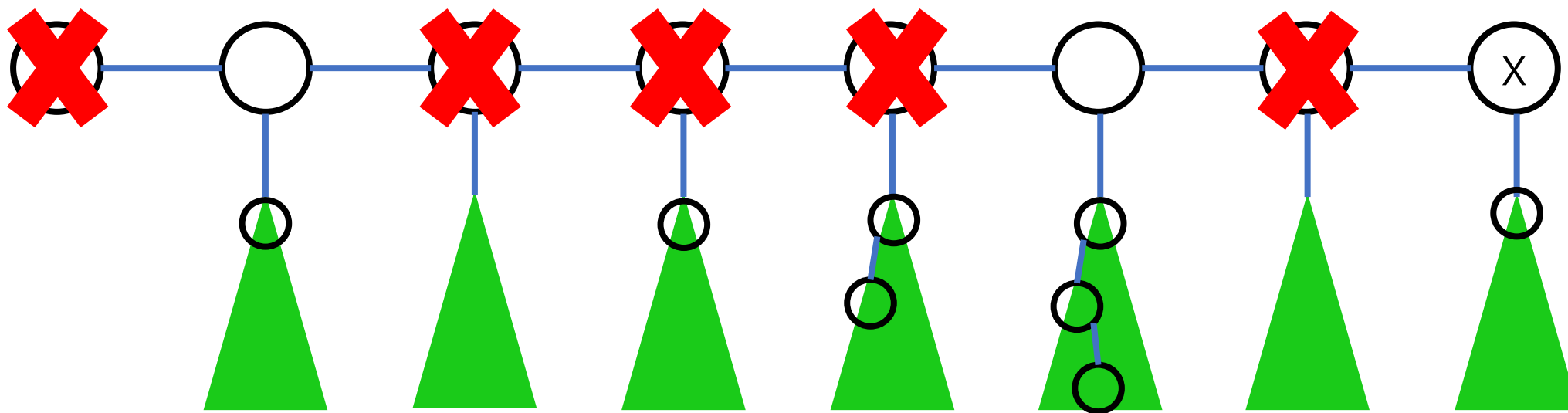
この3つの頂点は珍しい都市の候補から消える



例えばここに長さ3のパスがあれば

# 考察

- 最後に残った頂点は、すべて珍しい頂点



# 得点分布

- 小課題①の正答者      ?1人
- 小課題②の正答者      ??人
- 小課題③の正答者      ??人
- 小課題④の正答者      0?人

# 考察

- ところで、木の直径の端点（最も距離の長い2点对）を任意にとって  $D1, D2$  とおく
- 任意の頂点  $x$  について、 $D1, D2$  の少なくとも一方は、 $x$  からの距離最大の頂点



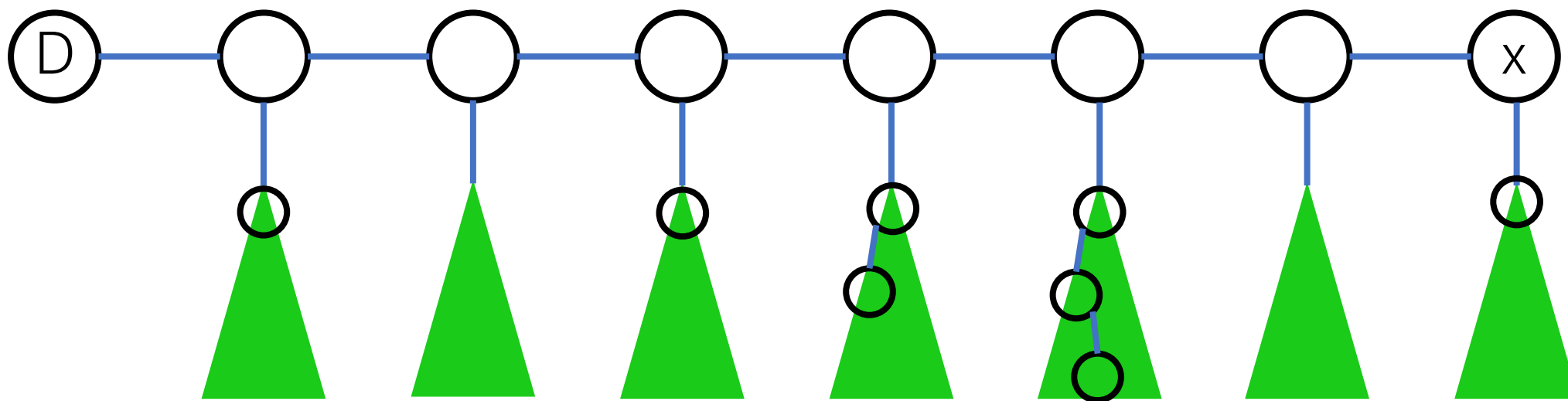
# 考察

- 頂点  $D1$  から DFS をして、各頂点  $x$  について、 $D1-x$  パス上に珍しい頂点が存在するかどうか判定することができればよい
- $D2$  についても同じ DFS をすれば、各頂点について正しい答えが求まる

# 考察

- D-x パス上にある珍しい頂点は、各部分木の最大深さの列からわかる

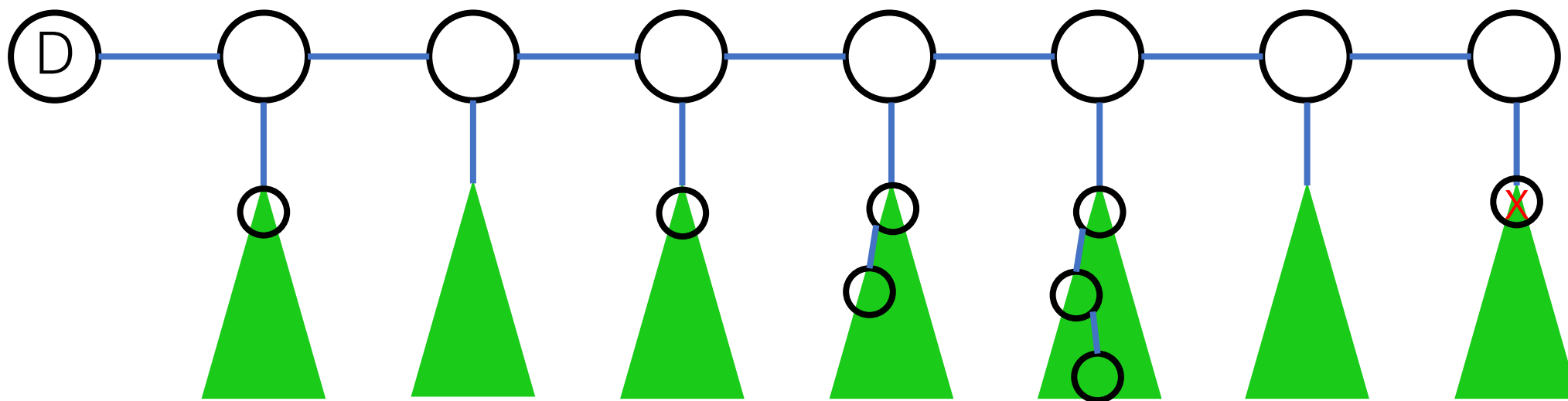
例えば以下の例では (0,1,0,1,2,3,0,1)



# 考察

- この列はDFSで辺を一本通るたびに、後ろの定数個の要素が変化する

例えば以下の例では  $(0,1,0,1,2,3,0,1) \rightarrow (0,1,0,1,2,3,0,0)$



# 考察

- この列を  $A$  とおくと問題は次のようになる
- $A$  を変化させるクエリ（ただし末尾の要素のみ）がたくさん与えられる
- 各  $p (1 \leq p \leq |A|)$  について、 $p - A[p] \leq j < p$  を満たす  $j$  を使用禁止にしたとき、禁止されない  $|A|$  未満の  $j$  の集合を  $S$  とおく
- $S$  の要素が珍しい頂点に対応

# 得点分布

- 小課題①の正答者      ?1人
- 小課題②の正答者      ??人
- 小課題③の正答者      0?人
- 小課題④の正答者      0?人

## 小課題②

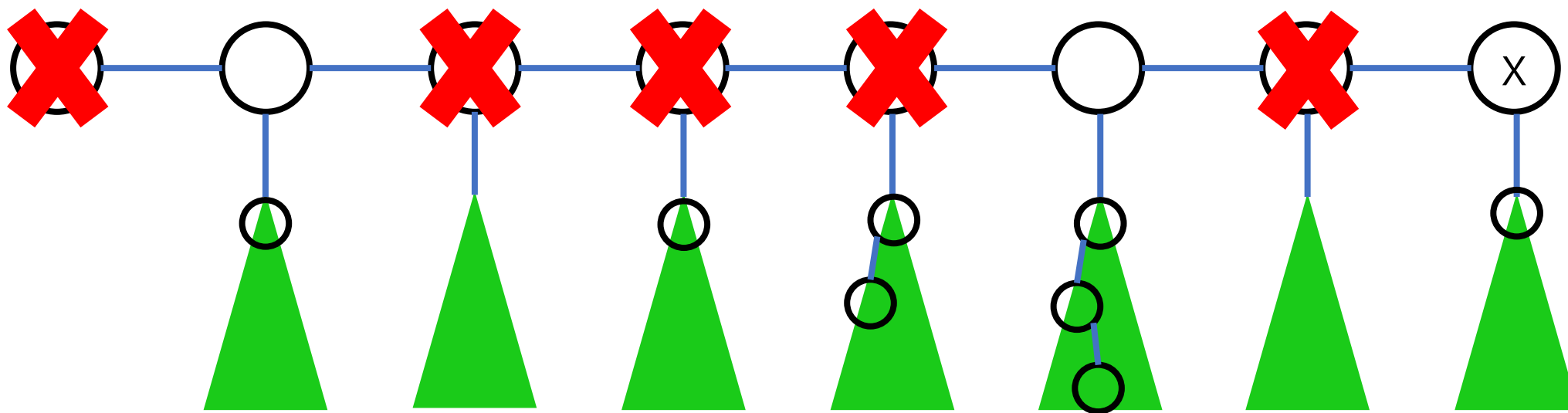
- 頂点のラベルが 1 種類
- 珍しい頂点が存在するかどうか判定すればいい
- $S$  が空かどうか判定すればいい

## 小課題②

- 頂点  $x$  に来た段階では、 $D-x$  パスのうち、 $x$  以外から生える部分木の最大深さの列を保持しておくことにする（つまり  $A$  の末尾を削ったものを保持しておく）
- これを  $A'$  とおく
- $S$  の  $\min$  と、 $x$  から生える部分木の最大深さの比較によって、答えが求まる

## 小課題②

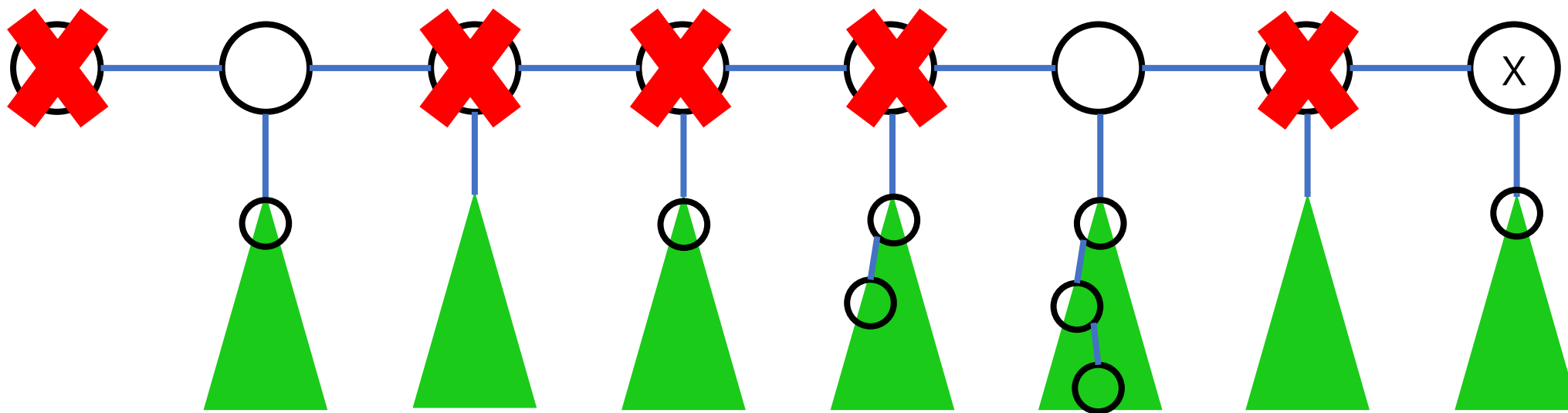
- 例えば以下の例では  $A' = (0, 1, 0, 1, 2, 3, 0)$ ,  $\min\{S\} = 2$





## 小課題②

- $x$  から生える部分木の最大深さが6以上だと  $S$  は空になり、そうでないなら  $\min$  の要素が残るため空ではない



## 小課題②

- あらかじめ各部分木の最大深さを求めておく
- 辺を下るときに $A'$ の末尾に追加されるのは、今下った部分木の兄弟の部分木の最大深さ
- $S$ のminは、そのままか、 $|A'|+1$ になるかどうか

## 小課題②

- 辺を一回下る操作は  $O(1)$  でできる
- 全体で  $O(N)$
- ここまでで 36 点

# 得点分布

- 小課題①の正答者      ?1人
- 小課題②の正答者      0?人
- 小課題③の正答者      0?人
- 小課題④の正答者      0?人

## 小課題③

- ラベルがすべて異なる
- $|S|$ のサイズがわかればいい

## 小課題③

- ラベルがすべて異なる
- $|S|$ のサイズがわかればいい

## 小課題③

- 「各  $p(1 \leq p \leq |A|)$  について、  
 $p - A[p] \leq j < p$  を満たす  $j$  を使用禁止」とあるが、補助配列  $B$  を用意しておき、  
「各  $p(1 \leq p \leq |A|)$  について、  
 $p - A[p] \leq j < p$  を満たす  $j$  で  $B[j]++$ 」  
とし、 $B[j] = 0$  となる  $j$  を数えればいい

## 小課題③

- A の要素の変更は、B に対する区間加算クエリに対応
- B に対する区間加算と、0になる要素のカウントが高速にできればいい



## 小課題③

- B の要素は必ず0以上だから、 $\min\{B\}=0$  かどうかのチェックと、minの要素のカウントができればいい
- これはSegment Treeでできる
- 全く同じSegTreeが過去のIOIでも出題

## 小課題③

- Segment Tree の更新は 1 回  $O(\log N)$  でできる
- 更新は  $O(N)$  回なので、全体で  $O(N \log N)$
- ここまでで 68 点

# 得点分布

- 小課題①の正答者 21人
- 小課題②の正答者 0?人
- 小課題③の正答者 0?人
- 小課題④の正答者 0?人

## 小課題④

- A を変化させるクエリ（ただし末尾の要素のみ）

をもう少し詳しく見てみる

## 小課題④

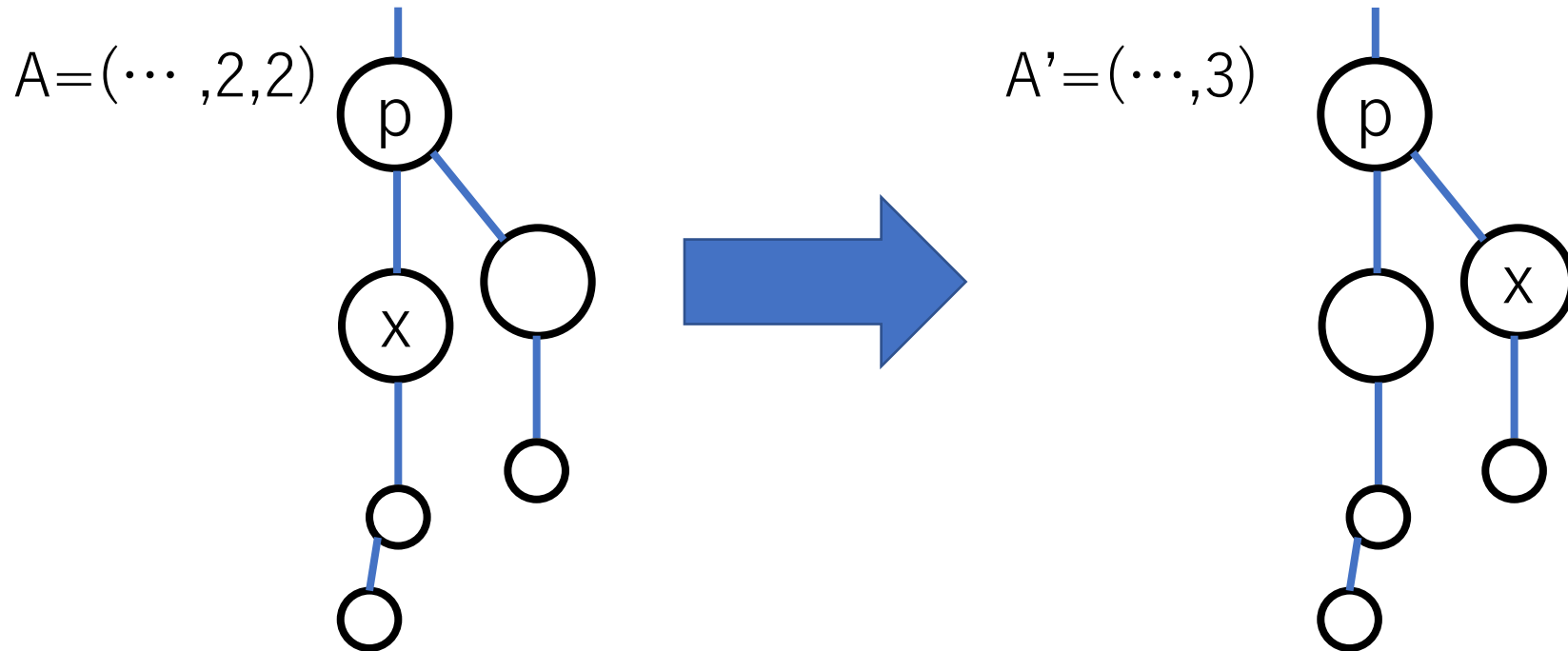
- 頂点  $x$  に来た段階では、 $A'$  を保持しており、  
頂点  $x$  から親に戻る段階では、 $A$  を保持しているようにする

## 小課題④

- 「辺を下るときに $A'$ の末尾に追加されるのは、今下った部分木の兄弟の部分木の最大深さ」とあるが、部分木の最大深さの後順に潜ることで、 $A'$ の末尾に追加される要素は単調増加する

## 小課題④

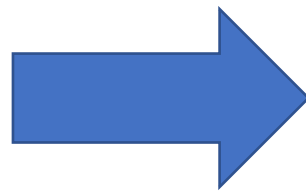
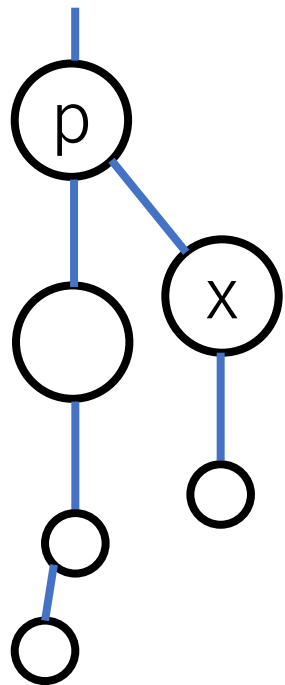
- DFSで兄弟の部分木に移るとき、 $A$ の末尾の要素の変化を見ると、 $p$ 以外の要素は $S$ から消えることはあっても増えることはない



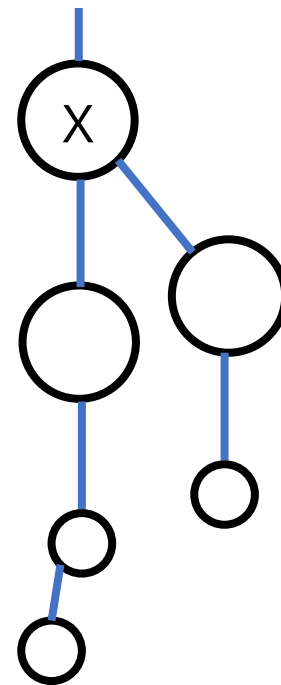
## 小課題④

- 親の頂点に戻るとき、 $A$ の末尾の要素の変化をみると、 $S$ の要素は消えることはあっても増えることはない

$A=(\dots, 3, 1)$



$A=(\dots, 3)$





## 小課題④

- 以上より、 $S$ の要素が増える回数は $O(N)$ 回、  
よって減る回数も $O(N)$ で抑えられる
- $S$ に対する操作はstackで表現できるため、1  
回あたり $O(1)$

## 小課題④

- Sの要素が増減するたびに、対応する頂点のラベルの集合をいじって、ラベルの種類数を更新すればいい
- 全体で $O(N)$
- 100 点

# 得点分布

- 小課題①の正答者 21人
- 小課題②の正答者 01人
- 小課題③の正答者 0?人
- 小課題④の正答者 0?人

# 得点分布

- 小課題①の正答者 21人
- 小課題②の正答者 01人
- 小課題③の正答者 0?人
- 小課題④の正答者 00人

# 得点分布

- 小課題①の正答者 21人
- 小課題②の正答者 01人
- 小課題③の正答者 00人
- 小課題④の正答者 00人